□> **Templates :** C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_01.pdf

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€" Year 1 â€" English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

By the end of this session, the student will be able to:

• Describe the overall module structure, its 10 sessions and general objectives

• Explain why computer science is essential in the training of English teachers

• Navigate the assessment methods (TD, TP, MCQ, project)

• Link each course topic to high school CS curricula (SNT/NSI)

• Identify the digital competencies (CRCN / PIX) expected of a teacher

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€" defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€" sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€" determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€" national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€" comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. Module presentation â€" plan, objectives, assessment *(15 min)*

2. Round table: students' representations of computer science *(10 min)*

3. The 10 sessions of the module â€" timeline and dependency graph *(10 min)*

4. Why computer science for language teachers? *(20 min)*

5. Links with high school curricula SNT and NSI *(15 min)*

6. Detailed assessment methods *(10 min)*

7. Questions and expectations *(10 min)*

1. Module Presentation

```
mindmap
root((Computer Science<br/>ENS Year 1))
Architecture
Info Representation
Binary / Hex
ASCII / UTF-8
Bitmap images
Machine
von Neumann
Fetch-Decode-Execute
Memory hierarchy
OS
Processes & threads
Scheduling
File system
Algorithms
Fundamentals
Variables & types
Conditions
Loops
Complexity
O(1), O(n), O(nÂ²)
Structures
Arrays
Stacks & queues
Networks & Web
TCP/IP & DNS
HTTP/HTTPS
Client-server model
Security & Ethics
CIA triad
Passwords & MFA
GDPR & student data
Python Programming
Variables & types
Conditions & loops
Functions & files
Educational project
```

General module objectives:

• Understand the fundamental architecture of a computer system (CPU, memory, I/O, buses)

• Master the basics of operating systems (processes, threads, files, commands)

• Design algorithms and evaluate their elementary complexity (O(1), O(n), O(nÂ²))

• Use classic data structures (arrays, LIFO stacks, FIFO queues)

• Understand networking principles (IP, DNS, HTTP/HTTPS, client-server)

• Master the basics of computer security (CIA triad, passwords, MFA, GDPR)

• Get started with Python programming for teaching purposes

• Link each concept to high school CS curricula (SNT/NSI)

Prerequisites: Baccalaureate; basic ICT skills (web browser, word processor, email).

Target audience: Year 1 students at the Ã‰cole Normale SupÃ©rieure, English Department, future secondary school English teachers.

2. The 10 Sessions of the Module

```
timeline
title Course timeline
Session 01 : Module introduction : objectives, plan, assessment
Session 02 : Information representation : binary, hex, ASCII, UTF-8, images
Session 03 : Machine architecture : von Neumann, fetch-decode-execute, memory hierarchy
Session 04 : Operating systems : processes, threads, scheduling, files
Session 05 : Fundamental algorithms : variables, conditions, loops, complexity
Session 06 : Data structures : arrays, LIFO stacks, FIFO queues, linked lists
Session 07 : Networks and Internet : TCP/IP, DNS, HTTP/HTTPS, client-server
Session 08 : Computer security : confidentiality, integrity, availability, GDPR
Session 09 : Introduction to Python : variables, conditions, loops, functions, files
Session 10 : Python educational project : quiz, grade calculator, mini-application

graph LR
S01[Session 01<br/>Introduction] --> S02[Session 02<br/>Representation]
S02 --> S03[Session 03<br/>Architecture]
S03 --> S04[Session 04<br/>Operating Systems]
S04 --> S05[Session 05<br/>Algorithms]
S05 --> S06[Session 06<br/>Data Structures]
S06 --> S07[Session 07<br/>Networks]
S07 --> S08[Session 08<br/>Security]
S08 --> S09[Session 09<br/>Python]
S09 --> S10[Session 10<br/>Python Project]

style S01 fill:#4a90d9,color:#fff
style S05 fill:#2ecc71,color:#fff
style S10 fill:#e74c3c,color:#fff
```

Pedagogical progression: Sessions 02 to 04 lay the theoretical foundations of machine architecture. Sessions 05 and 06 introduce algorithms and data structures, the bedrock of computational thinking. Sessions 07 and 08 cover networks and security, essential citizenship skills. Finally, sessions 09 and 10 are dedicated to Python programming and the end-of-module project.

3. Why Computer Science Matters for Language Teachers

Observation: Digital technology is omnipresent in language teaching: LMS platforms (Moodle, Google Classroom), spell-checkers (Antidote, LanguageTool), online dictionaries (WordReference, Linguee), exercise builders (Quizlet, LearningApps, Duolingo), virtual classrooms (Zoom, BigBlueButton), collaborative writing tools (Etherpad, Framapad), and generative AI (ChatGPT for writing).

Why must an English teacher master the basics of computer science?

1. **Contribution to high school CS curricula:** Digital literacy is mandatory for all students. English teachers can be involved in topics such as "Structured data", "Internet", and "Social media" in connection with reading, writing, and information literacy.

2. **Digital competency frameworks:** The CRCN (French Digital Competency Framework) defines 5 areas (Information and data, Communication, Content creation, Protection and security, Digital environment). The PIX certification is mandatory for teachers. An English teacher must notably master the "Content creation" area (word processing, multimedia presentations, document editing).

3. **Digital pedagogical tools for the English classroom:** A trained teacher can integrate interactive quizzes (Kahoot, QuiziniÃ¨re), mind maps (XMind, Mindomo), online dictionaries and correction tools, collaborative writing platforms (Framapad, Etherpad), grammar exercise builders, and audio/video resources (Podcast, YouTube).

4. **Computational thinking and linguistic analysis:** Algorithmic logic (sequences, conditions, loops) is transposable to the study of grammar (top-down parsing, derivation trees), phonetics (conditional pronunciation rules), and language didactics (sequential structuring of learning).

5. **Digital citizenship:** Understanding how the Internet works, search engines, recommendation algorithms, personal data security, and the ethical issues of digital technology (GDPR, disinformation, cyberbullying) is essential for training students in responsible digital citizenship.

Concrete examples for the English classroom:

• Understanding why certain characters (Ã¦, Å“, Ã©, Ã¨) display incorrectly in a text file â†' UTF-8 encoding concept

• Analysing how a spell-checker works â†' string comparison algorithms

• Understanding Google search ranking â†' PageRank algorithm, indexing

• Securing student data â†' encryption concepts, passwords, GDPR

4. Links with High School Curricula

SNT Theme (Grade 10)	Course link	English classroom applications
Structured data	Session 02 (encoding) + Session 06 (structures)	Text typology, grammatical analysis
Internet	Session 07 (networks)	Research skills, source evaluation
Machine architecture	Session 03 (von Neumann)	History of technology, impact on writing
Algorithms	Session 05 (algorithms)	Narrative structures, generative grammar
Social media	Session 08 (security, ethics)	Media literacy, cyberbullying

NSI Theme (Grades 11-12)	Course link
Data types, operators	Session 05
Conditions and loops	Session 05
Arrays, stacks, queues	Session 06
Algorithmic complexity	Session 05
Data transmission	Session 07

5. Assessment Methods

```
pie title Module assessment breakdown
"TD (40%)" : 40
"Python Lab (35%)" : 35
"MCQ (25%)" : 25
```

Component	Detail	Weight
Tutorial (TD)	Paper exercises: architecture, networks, algorithms, structures	40 %
Python Lab (TP)	Practical sessions (09 and 10): programming in the computer lab	35 %
MCQ	5 questions per session (formative self-assessment)	25 %

Rules:

• The TD grade includes exercises to hand in after each session (except session 01)

• The Python Lab leads to an individual or pair project presented in session 10

• MCQs are graded out of 5 points each; only the 5 best scores count

• A short defence (5 minutes) of the Python project is scheduled for session 10

• Attendance is mandatory (continuous assessment)

MCQ Self-Assessment

1. **How many sessions does the Computer Science module include?**

a) 8 sessions b) 10 sessions âœ“ c) 12 sessions

2. **What is the weight of Python Lab in the total assessment?**

a) 25 % b) 35 % âœ“ c) 40 %

3. **Which French high school programme is linked to computer science in Grade 10?**

a) NSI b) SNT âœ“ c) EMI

4. **What is the main objective of the module for future teachers?**

a) Become a professional programmer b) Understand computer science in order to teach it âœ“ c) Repair computers

5. **Which digital tool is cited as an exercise builder for language teaching?**

a) Photoshop b) Quizlet âœ“ c) Excel

Conclusion

This introductory session has set the framework for the module. Computer science is not an isolated discipline: it is at the heart of modern pedagogical practices, including in language teaching. The 10 sessions that follow are designed to give future English teachers the fundamental concepts and practical skills needed to integrate digital technology into their teaching and to contribute effectively to high school CS curricula.

Next session: Information representation â€” binary, hexadecimal, ASCII/UTF-8 encoding, bitmap image encoding.

Pedagogical Note ENS

Teaching transposition: This introductory session can be adapted for a Grade 10 class at the beginning of the school year. The teacher can:

• Conduct a round table on students' digital habits (smartphones, social media, video games)

• Present the CS curriculum in the form of a mind map (as done here)

• Explain the link between computer science and other disciplines (English, history, mathematics)

• Propose a reflection activity: "What is a computer useful for in daily life?"

Trainer advice: Invite ENS students to reflect on their own relationship with digital technology (strengths, weaknesses, expectations) to personalise their learning path throughout the module. Students can keep a digital learning journal of their progress.

Extension: Ask students to prepare for session 2 a short text (5 lines) listing the digital devices they use daily, to initiate reflection on information representation.

Bibliography

• French Ministry of Education (2019). *SNT Programme Grade 10* â€" Official Bulletin.

• French Ministry of Education (2019). *NSI Programme Grades 11-12* â€" Official Bulletin.

• CRCN â€" Digital Competency Reference Framework (2020). https://pix.fr/cadre-de-reference

• Dowek, G. et al. (2019). *Sciences NumÃ©riques et Technologie â€" Seconde*. Hachette Ã‰ducation.

• Tanenbaum, A. & Austin, T. (2012). *Structured Computer Organization* (6th ed.). Pearson.

• Python 3 Documentation (2024). https://docs.python.org/3/

• CSTA K-12 Computer Science Standards (2020). https://www.csteachers.org/

• Fluckiger, C. (2019). *Digital Technology and Language Teacher Training*. Revue des sciences de l'Ã©ducation.

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx` | `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_02.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 02 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

By the end of this session, the student will be able to:

• Convert a decimal number to binary and vice versa

• Convert a binary number to hexadecimal and vice versa

• Explain the difference between ASCII and Unicode/UTF-8

• Encode a character from its ASCII or Unicode code

• Calculate the size of a bitmap image (pixels, resolution, color depth)

• Apply these concepts to concrete examples for language teaching

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. Introduction: why represent information? *(5 min)*
2. The binary system (base 2) *(20 min)*
3. The hexadecimal system (base 16) *(15 min)*
4. Character encoding: ASCII, Unicode, UTF-8 *(20 min)*
5. Image encoding: bitmap, pixels, resolution, color *(20 min)*
6. Pedagogical applications for language teaching *(10 min)*

1. Coding Tree

```
graph TB
INFO["Digital Information"] --> NOMBRES["Numbers"]
INFO --> TEXTES["Texts"]
INFO --> IMAGES["Images"]
INFO --> SON["Sound (sampling)"]

NOMBRES --> BIN["Binary (base 2)<br/>0 and 1"]
NOMBRES --> DEC["Decimal (base 10)<br/>0-9"]
NOMBRES --> HEX["Hexadecimal (base 16)<br/>0-9, A-F"]

TEXTES --> ASCII["ASCII<br/>128 char. (7 bits)"]
TEXTES --> UNICODE["Unicode<br/>~150,000 char."]
UNICODE --> UTF8["UTF-8<br/>1-4 bytes"]

IMAGES --> BITMAP["Bitmap<br/>Pixel matrix"]
BITMAP --> COULEUR["RGB Color<br/>24 bits/pixel"]
BITMAP --> NB["Black & White<br/>1 bit/pixel"]

SON --> ECH["Sampling<br/>kHz + quantization"]

style BIN fill:#e74c3c,color:#fff
style HEX fill:#e67e22,color:#fff
style ASCII fill:#3498db,color:#fff
style UTF8 fill:#2ecc71,color:#fff
style BITMAP fill:#9b59b6,color:#fff
```

Digital information is always represented using **0s and 1s** (bits). Any type of data (number, text, image, sound, video) is converted to binary to be processed by a computer.

2. The Binary System (Base 2)

Principle

The binary system uses only two digits: **0** and **1**. Each position corresponds to a power of 2.

2â□·	2â□¶	2â□µ	2â□´	2Â³	2Â²	2Â¹	2â□°
128	64	32	16	8	4	2	1

Decimal â† Binary Conversion: Successive Division Method

```
Example: converting 42 to binary

42 Ã· 2 = 21, remainder 0 (least significant bit)
21 Ã· 2 = 10, remainder 1
10 Ã· 2 = 5, remainder 0
5 Ã· 2 = 2, remainder 1
2 Ã· 2 = 1, remainder 0
1 Ã· 2 = 0, remainder 1

Reading remainders from bottom to top: 42â,□â,€ = 101010â,,
```

Binary â† Decimal Conversion: Powers Method

```
101010â,, = 1Ã—2â□µ + 0Ã—2â□´ + 1Ã—2Â³ + 0Ã—2Â² + 1Ã—2Â¹ + 0Ã—2â□°
= 32 + 0 + 8 + 0 + 2 + 0
= 42â,□â,€

graph LR
subgraph "Binary â† Decimal"
```

```
B1["1010â,,"] --> P1["1Ã—2Â³ + 0Ã—2Â² + 1Ã—2Â¹ + 0Ã—2â°"]
P1 --> R1["= 8 + 0 + 2 + 0 = 10â,â,€"]
end

subgraph "Decimal â†’ Binary"
DEC1["13â,â,€"] --> DIV["13Ã·2=6 r1<br/>6Ã·2=3 r0<br/>3Ã·2=1 r1<br/>1Ã·2=0 r1"]
DIV --> BIN1["= 1101â,,"]
end

style R1 fill:#2ecc71,color:#fff
style BIN1 fill:#2ecc71,color:#fff
```

Bit = Binary Digit. A byte = 8 bits. A byte can encode 256 different values (2â¸).

3. The Hexadecimal System (Base 16)

Principle

Hexadecimal uses 16 digits: **0-9** then **A-B-C-D-E-F** (A=10, B=11, C=12, D=13, E=14, F=15).

Why hexadecimal? Because one hex digit corresponds exactly to 4 bits (a nibble), which makes binary-hex conversions very simple.

Conversion Table

```
block-beta
columns 4
block:group1
columns 4
B0["0000"] B1["0001"] B2["0010"] B3["0011"]
B4["0100"] B5["0101"] B6["0110"] B7["0111"]
B8["1000"] B9["1001"] BA["1010"] BB["1011"]
BC["1100"] BD["1101"] BE["1110"] BF["1111"]
end
space
block:group2
columns 4
H0["0"] H1["1"] H2["2"] H3["3"]
H4["4"] H5["5"] H6["6"] H7["7"]
H8["8"] H9["9"] HA["A"] HB["B"]
HC["C"] HD["D"] HE["E"] HF["F"]
end
space
block:group3
columns 4
D0["0"] D1["1"] D2["2"] D3["3"]
D4["4"] D5["5"] D6["6"] D7["7"]
D8["8"] D9["9"] D10["10"] D11["11"]
D12["12"] D13["13"] D14["14"] D15["15"]
end
group1 --> group2 --> group3
```

Binary â†’ Hex conversion: Group by 4 bits from the right, convert each group.

```
1010 1110 0101â,, â†’ A E 5â,â,† â†’ AE5â,â,†

A (1010) = 10â,â,€
E (1110) = 14â,â,€
5 (0101) = 5â,â,€
```

Hex â†’ Binary conversion: Each hex digit yields 4 bits.

```
3Fâ,â,† â†’ 0011 1111â,, â†’ 111111â,,
```

Hex â†’ Decimal conversion:

```
AE5â,□â,† = AÃ—16Â² + EÃ—16Â¹ + 5Ã—16â□°
= 10Ã—256 + 14Ã—16 + 5
= 2560 + 224 + 5
= 2789â,□â,€
```

Common usage: Hexadecimal is used to represent colors in HTML/CSS (`#FF0000` for red, `#FFFFFF` for white, `#000000` for black), memory addresses, and MAC addresses.

4. Character Encoding

ASCII (American Standard Code for Information Interchange)

• Created in 1963, standardized in 1967

• **7 bits** per character â†’ 128 characters (0-127)

• Contains: English letters (A-Z, a-z), digits (0-9), punctuation marks, control characters

```
graph TD
subgraph "ASCII Table (excerpt)"
L1["65 â†’ A<br/>66 â†’ B<br/>67 â†’ C"]
L2["97 â†’ a<br/>98 â†’ b<br/>99 â†’ c"]
L3["48 â†’ 0<br/>49 â†’ 1<br/>50 â†’ 2"]
L4["32 â†’ space<br/>33 â†’ !<br/>63 â†’ ?"]
end

subgraph "Limitation"
LIM["No accents<br/>No Ã§, Ã©, Ã¨, Ãª, Å“, Ã¦<br/>No Arabic, Chinese, etc."]
end

L1 --> LIM
L2 --> LIM
L3 --> LIM
L4 --> LIM
```

Example: The word "Hello" in ASCII:

```
H â†’ 72 â†’ 0100 1000
e â†’ 101 â†’ 0110 0101
l â†’ 108 â†’ 0110 1100
l â†’ 108 â†’ 0110 1100
o â†’ 111 â†’ 0110 1111
```

Unicode and UTF-8

Unicode: Universal character repertoire. Each character has a unique **code point** (e.g., U+0041 for 'A', U+00E9 for 'Ã©', U+0628 for 'Ø¨').

UTF-8: Dominant encoding on the Web (> 95%). Uses **1 to 4 bytes** per character:

• 1 byte: ASCII characters (U+0000 to U+007F) â€” backward compatibility

• 2 bytes: accented letters, Greek, Cyrillic, Arabic alphabets (U+0080 to U+07FF)

• 3 bytes: Asian characters (CJK), various symbols (U+0800 to U+FFFF)

• 4 bytes: rare characters, emojis (U+10000 to U+10FFFF)

Examples: A (U+0041, 1 byte 41), Ã© (U+00E9, 2 bytes C3 A9), Ã§ (U+00E7, C3 A7), Å“ (U+0153, C5 93), Ø¨ (U+0628, D8 A8), ðŸ˜€ (U+1F600, 4 bytes F0 9F 98 80).

Application for French language teaching: Accented characters (Ã©, Ã¨, Ãª, Ã«, Ã , Ã¢, Ã§, Å“, Ã¹, Ã®, Ã¯, Ã´, Ã», Ã¼) are encoded on 2 bytes in UTF-8. A text file containing French must be saved in UTF-8 to avoid display problems (appearance of "??" or unreadable characters).

5. Bitmap Image Encoding

Bitmap Image Diagram

```
graph TB
subgraph "Bitmap image 4Ã—4"
P11["(255,0,0)"] P12["(255,0,0)"] P13["(0,0,255)"] P14["(0,0,255)"]
P21["(255,0,0)"] P22["(255,0,0)"] P23["(0,0,255)"] P24["(0,0,255)"]
P31["(0,255,0)"] P32["(0,255,0)"] P33["(255,255,0)"] P34["(255,255,0)"]
P41["(0,255,0)"] P42["(0,255,0)"] P43["(255,255,0)"] P44["(255,255,0)"]
end

subgraph "Legend"
R["Red = (255,0,0)<br/>RGB 24 bits"]
B["Blue = (0,0,255)"]
G["Green = (0,255,0)"]
J["Yellow = (255,255,0)"]
end

style R fill:#ff0000,color:#fff
style B fill:#0000ff,color:#fff
style G fill:#00ff00,color:#000
style J fill:#ffff00,color:#000
```

Pixel = Picture Element. Smallest unit of a digital image.

Resolution: number of pixels (width Ã— height). Example: 1920 Ã— 1080 = 2,073,600 pixels (Full HD).

Color depth: number of bits per pixel (bpp).

• 1 bpp: black and white (2 colors)

• 8 bpp: 256 colors (palette)

• 24 bpp: 16.7 million colors (RGB: 8 bits per channel)

• 32 bpp: 24 bits + alpha channel (transparency)

Calculating Image Size

```
Size (in bytes) = Width Ã— Height Ã— (Depth / 8)

Example: 1920 Ã× 1080 image in 24 bits
Size = 1920 Ã× 1080 Ã× 3
= 6,220,800 bytes
â‰ˆ 5.93 MB
```

Exercise: An 800 Ã— 600 pixel image in black and white (1 bit/pixel) weighs:

```
800 Ã× 600 Ã× 1/8 = 60,000 bytes â‰ˆ 58.6 KB
```

Comparison: A 24-bit 12-megapixel photo (4000 Ã— 3000) weighs:

```
4000 Ã× 3000 Ã× 3 = 36,000,000 bytes = 36 MB (uncompressed)
```

This is why compressed formats (JPEG, PNG) are used.

MCQ Self-Assessment

1. **How many bits are there in a byte?**

a) 4 bits b) 8 bits âœ“ c) 16 bits

2. **What is the decimal value of 1010â‚‚?**

a) 8 b) 10 âœ“ c) 12

3. **Which character encoding is dominant on the Web?**

a) ASCII b) UTF-8 âœ“ c) ISO-8859-1

4. **How many bytes does UTF-8 use to encode the character Â« Ã© Â»?**

a) 1 byte b) 2 bytes âœ“ c) 3 bytes

5. **A 1024 Ã— 768 pixel image in 24 bits weighs approximately:**

a) 768 KB b) 2.25 MB âœ“ c) 5 MB

Conclusion

Information representation is the foundation of all computing. Binary/hex/decimal conversions, character encoding (ASCII, Unicode/UTF-8), and image encoding are essential concepts for understanding how a computer stores and manipulates data. For a French teacher, mastering UTF-8 encoding is particularly important to avoid display problems of accented characters in digital documents.

Next session: Machine architecture â€" the von Neumann model and the execution cycle.

Pedagogical Note ENS

Teaching transposition: In SNT Seconde (theme "Structured Data"), one can propose:

• Unplugged activity: encode your first name in binary using black (0) and white (1) cards

• Activity: convert French words to ASCII, then observe that accents are not encoded

• Activity: create a black-and-white bitmap image on graph paper (each square = one pixel)

Link with languages: Explain why certain French characters (Å“, Ã¦, Å', Ã†) can cause problems in files exported from Word to another software (encoding issue). Show the importance of always saving in UTF-8.

Trainer advice: To help students visualize abstract coding concepts, prioritize unplugged activities (cards, graph paper) before moving to digital tools. Use the "Binary Coding" website for interactive exercises. Remind that mastery of character encoding is essential for French teachers who handle text files with accents.

Extension: Students can deepen their knowledge by exploring image encoding (bitmap, RGB, JPEG compression) and audio encoding (sampling, quantization). Propose a complementary activity: create a bitmap image in Python from a grayscale pixel matrix (interdisciplinary mathematics-computer science project).

Bibliography

• Goupille, P. (2014). *Technologie des ordinateurs*. Ã‰ditions Dunod (chapters 1 and 2).

• Tanenbaum, A. & Austin, T. (2012). *Architecture de l'ordinateur* (6th ed.). Pearson.

• Tanenbaum, A. & Austin, T. (2012). *Structured Computer Organization* (6th ed.). Pearson.

• Unicode Consortium (2024). *The Unicode Standard* â€” https://unicode.org

• Dowek, G. et al. (2019). *SNT Seconde â€" Sciences NumÃ©riques et Technologie*. Hachette.

• Brookshear, J. G. (2014). *Computer Science: An Overview* (12th ed.). Pearson.

• Wikipedia â€" "Character encoding", "Bitmap", "UTF-8" (free consultation).

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx` | `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_03.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 03 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

By the end of this session, the student will be able to:

• Describe the von Neumann model and its 5 functional units

• Explain how the fetch-decode-execute cycle works

• Distinguish the different levels of the memory hierarchy (registers, cache, RAM, SSD, HDD)

• Calculate an average access time and compare performance

• Relate these concepts to SNT (architecture) and NSI (performance) programs

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. Introduction: what is a computer? *(5 min)*
2. The von Neumann model â€” the 5 units *(20 min)*
3. The fetch-decode-execute cycle *(20 min)*
4. The memory hierarchy â€” memory pyramid *(20 min)*
5. Performance comparison and practical calculations *(15 min)*
6. Pedagogical applications for SNT/NSI *(10 min)*

1. The von Neumann Model

```
graph TB
UC["Control Unit"] --> UAL["ALU"]
UC --> MEM["Memory<br/>(RAM)"]
UC --> E_S["Input / Output<br/>(I/O)"]

REG["Registers"] --> UAL
REG --> UC

MEM --> BUS["Data bus<br/>Address bus<br/>Control bus"]
BUS --> UC
BUS --> UAL
BUS --> E_S

subgraph "The 5 functional units"
U1["1. Control Unit"]
U2["2. ALU (Arithmetic & Logic)"]
U3["3. Memory (data + program)"]
U4["4. Input"]
U5["5. Output"]
end

style UC fill:#e74c3c,color:#fff
style UAL fill:#3498db,color:#fff
style MEM fill:#2ecc71,color:#fff
style E_S fill:#f39c12,color:#fff
style REG fill:#9b59b6,color:#fff
style BUS fill:#95a5a6,color:#fff
```

The von Neumann model (proposed by John von Neumann in 1945) is the fundamental architecture of almost all modern computers. Its 5 units are:

1.1 Control Unit

• Directs and coordinates all computer actions

• Reads instructions from memory and interprets them

• Generates control signals for other units

• Contains the **Program Counter** (PC) which points to the next instruction

• Contains the **Instruction Register** (IR) which stores the current instruction

1.2 Arithmetic and Logic Unit (ALU)

• Performs arithmetic operations: addition, subtraction, multiplication, division

• Performs logical operations: AND, OR, NOT, XOR

• Performs comparisons: equality, greater than, less than

• Uses **registers** (accumulators) to store operands and results

1.3 Memory (storage unit)

• Stores **both** data and programs (= von Neumann principle)

• Organized into addressable cells, each cell containing a memory word (8, 16, 32, or 64 bits)

• Two operations: **read** and **write**

1.4 Input / Output (I/O)

• **Input:** keyboard, mouse, microphone, camera, scanner, network

• **Output:** screen, speakers, printer, network

• Communication via **buses** (data, address, control)

1.5 Buses

• **Data bus:** carries data between units (bidirectional)

• **Address bus:** carries the memory address to read/write (unidirectional CPU â†' memory)

• **Control bus:** carries synchronization signals (read/write, clock)

Bus width: A 32-bit address bus can address 2Â³Â² = 4 GB of memory. A 64-bit bus can address 2â□¶â□´ = 16 EB (exabytes).

2. The Fetch-Decode-Execute Cycle

```
sequenceDiagram
participant PC as Program<br/>Counter (PC)
participant MEM as Memory (RAM)
participant IR as Instruction<br/>Register
participant UC as Control<br/>Unit
participant UAL as ALU

Note over PC,UAL: Instruction execution cycle

UC->>MEM: 1. FETCH: Send (PC) on address bus
MEM-->>UC: Read instruction at PC address
UC->>IR: Store instruction in IR
UC->>PC: PC â†□ PC + 1 (increment)

UC->>IR: 2. DECODE: Analyze opcode (e.g., ADD)
UC->>UC: Identify required operands

UC->>MEM: 3. FETCH Operands: Read operands
MEM-->>UAL: Transmit operands to ALU

UAL->>UAL: 4. EXECUTE: Perform operation
UAL-->>MEM: 5. STORE: Write result to memory
```

Detailed Steps

Step 1 â€" Fetch:

• The control unit places the content of the **PC** (program counter) on the address bus

• Memory returns the instruction located at that address

• The instruction is loaded into the **IR** (instruction register)

• The PC is incremented to point to the next instruction

Step 2 â€" Decode:

• The control unit decodes the **opcode** (operation code) to determine the operation to perform

• It identifies the **operands** (registers or memory addresses involved)

• It activates the appropriate circuits in the ALU

Step 3 â€" Execute:

• The ALU performs the operation (addition, subtraction, comparison, etc.)

• The result is temporarily stored in an accumulator register

Step 4 â€" Store:

• The result is written to a register or memory (RAM)

• The cycle repeats with the next instruction (pointed to by PC)

```
graph TD
DEBUT("Start") --> FETCH["FETCH<br/>Load instruction<br/>from RAM to IR"]
FETCH --> DECODE["DECODE<br/>Analyze opcode<br/>in control unit"]
DECODE --> EXECUTE["EXECUTE<br/>Perform operation<br/>in ALU"]
EXECUTE --> STORE["STORE<br/>Write result<br/>to memory/register"]
STORE --> FETCH

style DEBUT fill:#2ecc71,color:#fff
style FETCH fill:#3498db,color:#fff
style DECODE fill:#e67e22,color:#fff
style EXECUTE fill:#e74c3c,color:#fff
style STORE fill:#9b59b6,color:#fff
```

Clock frequency: Each cycle is clocked by the processor's clock. A 3 GHz CPU performs 3 Ã— 10â□¹ cycles per second. A simple instruction (ADD) may take 1 cycle, a complex instruction (DIV) may take several.

3. The Memory Hierarchy

Memory Pyramid

```
graph TB
subgraph "Memory hierarchy (decreasing speed, increasing capacity)"
R1["Registers<br/>~ 1 KB<br/>~ 0.3 ns<br/>< 1 cycle"]
R2["Cache L1<br/>~ 32-64 KB<br/>~ 1 ns<br/>3-4 cycles"]
R3["Cache L2<br/>~ 256-512 KB<br/>~ 3 ns<br/>~ 10 cycles"]
R4["Cache L3<br/>~ 8-16 MB<br/>~ 10 ns<br/>~ 30 cycles"]
R5["RAM (DDR4/5)<br/>~ 8-64 GB<br/>~ 50-100 ns<br/>~ 150 cycles"]
R6["SSD (NVMe)<br/>~ 256 GB - 2 TB<br/>~ 10,000 ns<br/>~ 30,000 cycles"]
R7["HDD<br/>~ 500 GB - 10 TB<br/>~ 10,000,000 ns<br/>~ 30 M cycles"]
end

R1 --> R2 --> R3 --> R4 --> R5 --> R6 --> R7

style R1 fill:#e74c3c,color:#fff
style R2 fill:#e67e22,color:#fff
style R3 fill:#f1c40f,color:#000
style R4 fill:#2ecc71,color:#fff
style R5 fill:#3498db,color:#fff
style R6 fill:#9b59b6,color:#fff
style R7 fill:#34495e,color:#fff
```

The Different Memories

1. Registers:

• Integrated into the processor

• Capacity: a few dozen to a few hundred bytes

• Access time: ~ 0.3 ns (1 clock cycle)

• Usage: storing operands and immediate results of the ALU

2. Cache memory (L1, L2, L3):

• Fast and expensive memory, close to the CPU

• Stores frequently used data and instructions (principle of **spatial and temporal locality**)

• L1: fastest, dedicated per core

• L2: shared or per core, less fast

• L3: shared among all cores

3. RAM (Random Access Memory):

• Volatile memory (lost on power off)

• Capacity: 8 to 64 GB (depending on configuration)

• Access time: ~ 50 ns (DDR4)

• Organization: DIMM/SO-DIMM modules

4. SSD (Solid State Drive):

• Non-volatile memory (NAND Flash)

• Capacity: 256 GB to 2 TB

• Access time: ~ 10,000 ns (10 Âµs)

• Advantages: fast, silent, shock-resistant

• Drawback: limited write cycles

5. HDD (Hard Disk Drive):

• Non-volatile magnetic memory

• Capacity: 500 GB to 10+ TB

• Access time: ~ 10,000,000 ns (10 ms)

• Advantage: very low cost per GB

• Drawback: slow, mechanical, noisy

Principle of Locality

• **Temporal locality:** recently accessed data will likely be reused soon (loops)

• **Spatial locality:** data near accessed data will likely be accessed soon (arrays)

These principles explain cache efficiency: the CPU automatically copies frequently used data to cache, significantly speeding up execution.

Calculating Average Access Time

Example: cache hit rate = 95%, cache time = 1 ns, RAM time = 50 ns.

```
Average time = 0.95 Ã— 1 ns + 0.05 Ã× 50 ns = 0.95 + 2.5 = 3.45 ns
```

Without cache, the access time would be 50 ns. With a 95% hit rate cache, we gain a factor of 14.

MCQ Self-Assessment

1. **How many functional units make up the von Neumann architecture?**

a) 3 units b) 5 units âœ“ c) 7 units

2. **What does the acronym ALU stand for?**

a) Arithmetic and Logic Unit âœ“ b) Application Linear Unit c) Access Local Unit

3. **In the fetch-decode-execute cycle, what happens at the FETCH step?**

a) The instruction is executed by the ALU b) The instruction is loaded from RAM âœ“ c) The result is stored in memory

4. **What is the fastest memory in the hierarchy?**

a) Cache L3 b) RAM c) Registers âœ“

5. **An SSD is approximately how many times faster than an HDD?**

a) 2 times b) 100 times âœ“ c) 10,000 times

Conclusion

The von Neumann architecture is the universal model of all modern computers. The 5 units (control, ALU, memory, input, output) work in synergy through the fetch-decode-execute cycle. The memory hierarchy (registers â† cache â† RAM â† SSD â† HDD) is designed to balance speed and capacity: the further from the CPU, the slower and larger the memory. These concepts are fundamental to understanding the performance of a computer system.

Next session: Operating systems â€" processes, threads, scheduling, and files.

Pedagogical Note ENS

Teaching transposition: For teaching architecture in SNT Seconde (theme "Machine Architecture"):

• Unplugged activity: simulate the fetch-decode-execute cycle with students playing roles (control unit, memory, ALU)

• Build a physical memory pyramid with cups or boxes to visualize the hierarchy

• Compare access times with analogies: register = your brain (1 sec), cache = desk (3 sec), RAM = library (5 min), SSD = neighboring city (2h), HDD = distant country (weeks)

Link with languages: Draw a parallel between cache memory (temporary storage of frequent data) and active/passive vocabulary in language learning.

Trainer advice: Be careful not to overwhelm students with technical architecture details. The goal is for them to understand the path of an instruction (from click to display) and not the specific features of microprocessors. Use animated videos of the execution cycle (e.g., "How a CPU works" from the CrashCourse YouTube channel) to reinforce visual understanding.

Extension: Students can explore MIPS or RISC-V assembly using the online simulator "MARS" to visualize instruction-by-instruction execution. Propose a complementary activity: compare CISC (Intel) and RISC (ARM) architectures and their implications in mobile devices used in educational settings (tablets, smartphones).

Bibliography

• von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. Contract W-670-ORD-4926.

• Tanenbaum, A. & Austin, T. (2012). *Architecture de l'ordinateur* (6th ed.). Pearson.

• Tanenbaum, A. & Austin, T. (2012). *Structured Computer Organization* (6th ed.). Pearson.

• Hennessy, J. & Patterson, D. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.

• Goupille, P. (2014). *Technologie des ordinateurs*. Dunod.

• Dowek, G. et al. (2019). *SNT Seconde*. Hachette (chapter "Machine Architecture").

Templates *:* `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_04.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 04 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

By the end of this session, the student will be able to:

• Explain the role of an operating system and the kernel/user space distinction

• Define a process and distinguish it from a thread

• Describe the 3 states of a process (ready, running, waiting)

• Explain the principle of CPU scheduling (FIFO, Round Robin)

• Navigate a file tree and understand Unix permissions

• Adapt this knowledge for SNT activities and teaching situations

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. Introduction: what is an operating system? *(5 min)*

2. OS architecture: kernel, kernel space, user space *(15 min)*

3. Processes and threads: definitions, states, lifecycles *(20 min)*

4. CPU scheduling: FIFO, Round Robin, priorities *(20 min)*

5. File system: tree structure, directories, permissions *(20 min)*

6. Pedagogical applications and command line *(10 min)*

1. Role of the Operating System

```
graph TB
USER["User<br/>(applications)"] --> UI["User Interface<br/>(GUI / CLI)"]
UI --> OS["Operating System"]
OS --> KERNEL["Kernel"]

subgraph "Kernel services"
PROC["Process management<br/>Scheduling"]
MEM["Memory management<br/>Virtual memory"]
FS["File system<br/>Storage & permissions"]
DEV["Device management<br/>Drivers"]
NET["Network<br/>TCP/IP stack"]
end

KERNEL --> PROC
KERNEL --> MEM
KERNEL --> FS
KERNEL --> DEV
KERNEL --> NET

PROC --> HW["Hardware<br/>(CPU, RAM, disks, ...)"]
MEM --> HW
FS --> HW
DEV --> HW
NET --> HW

style OS fill:#e74c3c,color:#fff
style KERNEL fill:#3498db,color:#fff
style HW fill:#95a5a6,color:#fff
```

Definition: An operating system (OS) is software that interfaces between hardware and applications. It manages computer resources and provides services to programs.

Main roles:

1. **Hardware abstraction**: hide the complexity of hardware from applications
2. **Resource multiplexing**: share CPU, memory, disks among multiple programs
3. **Isolation and protection**: prevent one program from accessing another's data
4. **Input/Output management**: communicate with peripherals via drivers

OS examples: Windows 10/11 (Microsoft), Linux (Ubuntu, Debian, Fedora), macOS (Apple), Android (Google), iOS (Apple).

Kernel: core of the OS, responsible for resource management. It runs in **kernel mode** (full hardware access). Applications run in **user mode** (restricted access). System calls (syscalls) allow switching from user mode to kernel mode.

2. Processes and Threads

2.1 Definition of a Process

Process: a program in execution, with its own memory space (code, data, stack, heap), open files, and execution context.

Each process has:

• A **PID** (Process IDentifier) â€” unique number

• A **state** (ready, running, waiting)

• A **program counter** (PC) â€" next instruction to execute

• **Registers** (CPU context)

• An **open file table**

• A **private memory space**

2.2 Threads

Thread: a lighter execution unit than a process. A process can contain multiple threads that share the same memory space.

Process vs thread difference:

Characteristic	Process	Thread
Memory space	Independent (isolated)	Shared with threads of the same process
Creation	Heavier (resource copy)	Lighter
Communication	IPC (pipes, sockets, signals)	Direct shared memory
Isolation	Strong (a crash doesn't affect others)	Weak (a crash can destroy everything)
Switching	More costly	Faster

2.3 The 3 States of a Process

```
stateDiagram-v2
[*] --> NEW : Creation (fork)
NEW --> READY : Ready (loaded in memory)
READY --> RUNNING : Selected by<br/>the scheduler
RUNNING --> READY : Interrupt<br/>(time elapsed)
RUNNING --> WAITING : Waiting for a<br/>resource (I/O)
WAITING --> READY : Resource<br/>available
RUNNING --> TERMINATED : End of process (exit)
TERMINATED --> [*]

note right of READY : Waiting for CPU
note right of RUNNING : Currently executing
note right of WAITING : Blocked (I/O, sleep)
```

State transitions:

• **New â†' Ready:** the process is created and placed in the ready queue

• **Ready â†' Running:** the scheduler selects a ready process and allocates the CPU to it

• **Running â†' Ready:** the time quantum has expired, the process is returned to the ready queue

• **Running â†' Waiting:** the process requests an I/O operation (disk read, network wait)

• **Waiting â†' Ready:** the I/O operation is complete, the process returns to the ready queue

• **Running â†' Terminated:** the process has finished its execution (call to `exit()`)

Context switch: When the CPU switches from one process to another, it must save the context of the outgoing process (registers, PC) and restore that of the incoming process. This operation has a cost (a few microseconds).

3. CPU Scheduling

The **scheduler** decides which ready process gets the CPU. Objectives: fairness, efficiency, minimal response time.

3.1 FIFO (First In, First Out)

```
graph LR
subgraph "Ready process queue (FIFO)"
P1["P1<br/>Arrived at t=0"] --> P2["P2<br/>Arrived at t=2"]
P2 --> P3["P3<br/>Arrived at t=4"]
P3 --> P4["P4<br/>Arrived at t=6"]
end

subgraph "Execution on CPU"
EXEC["P1 (0-5) | P2 (5-8) | P3 (8-10) | P4 (10-12)"]
end

P1 --> EXEC
```

Principle: First process arrived is first served. No preemption (non-preemptive). Once a process has the CPU, it keeps it until it finishes or blocks.

Advantages: Simple to implement, fair for long processes.

Disadvantages: High average waiting time for short processes (convoy effect).

3.2 Round Robin (RR)

```
gantt
title Round Robin Scheduling (quantum = 2 units)
dateFormat X
axisFormat %s

section P1 (duration 6)
Round 1 : 0, 2
Round 2 : 6, 2
Round 3 : 10, 2

section P2 (duration 4)
Round 1 : 2, 2
Round 2 : 8, 2

section P3 (duration 2)
Round 1 : 4, 2

section P4 (duration 2)
Round 1 : 12, 2
```

Principle: Each process receives a fixed **time quantum** (e.g., 10 ms). If the process does not finish within its quantum, it is placed back in the ready queue and the next one is executed.

Advantages: Fair, short response time for interactive processes.

Drawback: The choice of quantum is crucial â€” too short â† too frequent context switching, too long â† degraded response time.

Calculation example (Round Robin, quantum = 2):

Process	Duration	Start	End	Waiting time
P1	6	0	12	6
P2	4	2	10	4
P3	2	4	6	2

P4	2	12	14	10

Average waiting time: (6 + 4 + 2 + 10) / 4 = 5.5 time units.

4. File System

4.1 File Tree Structure

```
graph TB
ROOT["/ (root)"] --> BIN["/bin<br/>essential commands"]
ROOT --> HOME["/home<br/>user directories"]
ROOT --> ETC["/etc<br/>system configuration"]
ROOT --> USR["/usr<br/>installed programs"]
ROOT --> VAR["/var<br/>variable data (logs)"]
ROOT --> TMP["/tmp<br/>temporary files"]
ROOT --> DEV["/dev<br/>devices"]

HOME --> USER1["/home/madani<br/>(your directory)"]
USER1 --> DOC["Documents"]
USER1 --> TEL["Downloads"]
USER1 --> BUR["Desktop"]
USER1 --> MUSIC["Music"]
USER1 --> IMG["Pictures"]

style ROOT fill:#e74c3c,color:#fff
style HOME fill:#3498db,color:#fff
style USER1 fill:#2ecc71,color:#fff
```

Structure: Rooted tree (Unix/Linux) or drive letters (Windows).

Absolute path: starts with `/` (root). Example: `/home/madani/Documents/course.md`

Relative path: relative to the current directory. Example: `../Documents/course.md`

Shortcuts:

- `.` : current directory
- `..` : parent directory
- `~` : user's home directory

4.2 Basic Commands (Linux terminal)

Command	Role	Example
pwd	Display current directory	pwd â†’ /hom
ls	List files	ls -la (detailed, hidden)
cd	Change directory	cd Documents/course
mkdir	Create a directory	mkdir new_folder
cp	Copy a file	cp source.txt dest.txt
mv	Move/rename	mv old.txt new.txt
rm	Delete	rm file.txt (careful!)
cat	Display content	cat file.txt
chmod	Change permissions	chmod 755 script.sh

4.3 Permissions (Unix)

```
graph TD
subgraph "File permissions"
PERM["-rwxr-xr--"]
TYP["Type:<br/>- file<br/>d directory<br/>l link"]
U["User (owner)<br/>rwx<br/>(read, write, execute)"]
G["Group<br/>r-x<br/>(read, execute)"]
O["Others<br/>r--<br/>(read only)"]
end

TYP --> PERM
PERM --> U --> G --> O
```

Symbolic representation: `-rwxr-xr--`

• Position 1: type (– file, `d` directory, `l` symbolic link)

• Positions 2-4: **owner** permissions (r=read, w=write, x=execute)

• Positions 5-7: **group** permissions

• Positions 8-10: **others** permissions

Octal representation:

• `r` = 4, `w` = 2, `x` = 1

• `chmod 755` â†' `rwxr-xr-x`

• `chmod 644` â†' `rw-r--r--`

Teaching example: Protecting a confidential grade file:

```
chmod 600 student_grades.txt # only owner can read/write
```

MCQ Self-Assessment

1. **What is the main role of the OS kernel?**

a) Display a graphical interface b) Manage hardware resources âœ“ c) Browse the Internet

2. **In which state does a process wait for the CPU to be allocated to it?**

a) Running b) Waiting c) Ready âœ“

3. **Which scheduling algorithm uses a fixed time quantum?**

a) FIFO b) Round Robin âœ“ c) SJF

4. **What does the `x` permission mean on a directory?**

a) Being able to delete it b) Being able to traverse (access) it âœ“ c) Being able to rename it

5. **Which command displays the current directory under Linux?**

a) `cd` b) `ls` c) `pwd` âœ“

Conclusion

The operating system is fundamental software that manages all computer resources. The kernel/user space distinction guarantees stability and security. Processes and threads are the basic execution units; their scheduling (FIFO, Round Robin) determines system responsiveness. The file system organizes data in a tree structure and Unix permissions control access.

Next session: Fundamental algorithms â€” variables, conditions, loops, and complexity.

Pedagogical Note ENS

Teaching transposition: In SNT Seconde (theme "Operating Systems"):

• Simulate Round Robin scheduling by having students play the roles of processes. One student is the scheduler and distributes a "quantum" (e.g., 30 seconds) to each process.

• Activity: create a directory tree for a class project (using `mkdir` on the command line)

• Explain permissions with the analogy of a classroom (owner = teacher, group = students)

Link with languages: The file tree concept can be compared to grammatical structure (syntax trees). Threads (lightweight execution) can be compared to parallel conversations in a classroom.

Trainer advice: Emphasize the difference between process and program (a concept often confused by beginners). Show the task manager (Windows) or system monitor (Linux) live during the course to illustrate active processes. Ask students to create a small Python script that lists system processes using the `psutil` library.

Extension: Students can explore Docker containers as an evolution of virtual machines, or study the Linux kernel and process management on the command line with `ps`, `top`, `htop`, `kill` tools. Propose a reflection on the impact of operating systems in education (choice between Windows, macOS, Linux, ChromeOS in schools).

Bibliography

• Tanenbaum, A. & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

• Silberschatz, A., Galvin, P. & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.

• The Linux Command Line: https://linuxcommand.org/

• Dowek, G. et al. (2019). *SNT Seconde*. Hachette (chapter "Operating Systems").

• Ubuntu Documentation: https://doc.ubuntu-fr.org/

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_05.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 05 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

By the end of this session, the student will be able to:

• Define an algorithm and list its 5 fundamental characteristics

• Use variables, data types, and expressions in pseudocode

• Write conditional structures (if/else/switch)

• Use conditional (while) and counted (for) loops

• Analyze the time complexity of a simple algorithm (O(1), O(n), O(nÂ²))

• Manually trace the execution of an algorithm step by step

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. What is an algorithm? Definition and characteristics *(10 min)*
2. Variables, types, and expressions *(15 min)*
3. Conditional structures: if, else, switch *(15 min)*
4. Loops: while, for *(20 min)*
5. Algorithmic complexity: O(1), O(n), O(nÂ²) *(15 min)*
6. Manual execution trace *(10 min)*
7. Pedagogical applications for SNT/NSI *(5 min)*

1. What is an Algorithm?

Definition: An algorithm is a **finite** sequence of **unambiguous** instructions that, given some **inputs**, produces **outputs** and **terminates** in a finite amount of time.

5 Fundamental Characteristics

1. **Finiteness:** the algorithm terminates after a finite number of steps (no infinite loop)
2. **Unambiguity:** each instruction is clear and without multiple interpretations
3. **Inputs:** zero or more input data
4. **Outputs:** one or more results produced
5. **Effectiveness:** the algorithm uses reasonable resources (time, memory)

Founding Example: Euclid's Algorithm (GCD)

Origin: From the Persian mathematician Al-KhwÃ¢rizmÃ® (9th century).

```
Algorithm GCD(a, b)
WHILE b â‰  0 DO
r â† a MOD b; a â† b; b â† r
END WHILE
RETURN a
```

```
graph TD
DEBUT("Start") --> ENTREE["Input a, b"]
ENTREE --> COND{"b = 0 ?"}
COND -- YES --> SORTIE["Display a (GCD)"]
SORTIE --> FIN("End")
COND -- NO --> CALC["r â† a MOD b<br/>a â† b<br/>b â† r"]
CALC --> COND

style DEBUT fill:#2ecc71,color:#fff
style ENTREE fill:#3498db,color:#fff
style COND fill:#f39c12,color:#000
style SORTIE fill:#3498db,color:#fff
style FIN fill:#e74c3c,color:#fff
style CALC fill:#9b59b6,color:#fff
```

2. Variables, Types, and Expressions

A **variable** is a named memory space containing a value that can change.

```
age â† 25
name â† "Alice"
is_adult â† TRUE
```

Type	Meaning	Examples
Integer (int)	Whole number	42, -3, 0
Real (float)	Decimal number	3.14, -0.5
Boolean (bool)	True or False	TRUE, FALSE
Character (char)	Single character	'A', 'Ã©'
String	Text	"Hello"

Operators:

- **Arithmetic:** +, −, Ã—, /, `MOD` (remainder), `DIV` (integer division)
- **Comparison:** =, â‰ , <, >, â‰¤, â‰¥
- **Logical:** `AND`, `OR`, `NOT`

```
average â† (grade1 + grade2 + grade3) / 3
is_even â† (x MOD 2 = 0)
adult â† (age â‰¥ 18) AND (nationality = "French")
```

3. Conditional Structures

3.1 If / Else

```
graph TD
COND2{"Condition ?"} -- TRUE --> B1["Instruction block 1"]
COND2 -- FALSE --> B2["Instruction block 2"]
B1 --> SUITE
B2 --> SUITE
style COND2 fill:#f39c12,color:#000
```

```
IF grade â‰¥ 10 THEN
DISPLAY "Passed"
ELSE
DISPLAY "Failed"
END IF
```

With ELSE IF:

```
IF grade â‰¥ 16: "Excellent" ; ELSE IF grade â‰¥ 14: "Good"
ELSE IF grade â‰¥ 12: "Fairly good" ; ELSE IF grade â‰¥ 10: "Passable" ; ELSE: "Failed"
END IF
```

3.2 Switch / Case

```
SWITCH grade DO
CASE "A": "Excellent" ; CASE "B": "Good" ; CASE "C": "Fairly good"
CASE "D": "Passable" ; CASE "E": "Insufficient" ; DEFAULT: "Invalid"
END SWITCH
```

4. Loops

4.1 WHILE Loop

Repeats a block of instructions **while** the condition is true. The test is performed before each iteration.

```
i â† 1
WHILE i â‰¤ 10 DO
DISPLAY i
i â† i + 1
END WHILE
```

Example â€” sum of the first N integers:

```
Algorithm Sum(N)
sum â† 0; i â† 1
WHILE i â‰¤ N DO
sum â† sum + i; i â† i + 1
END WHILE
RETURN sum
```

4.2 FOR Loop

Repeats a block a **determined** number of times. The counter is managed automatically.

```
FOR i FROM 1 TO N DO
DISPLAY "Iteration ", i
END FOR
```

Example â€” multiplication table:

```
Algorithm MultiplicationTable(N)
FOR i FROM 1 TO 10 DO
DISPLAY N, " Ã— ", i, " = ", N Ã— i
END FOR
```

```
graph TD
DEBUT2("Start") --> INIT["i â† 1"]
INIT --> COND3{"i â‰¤ N ?"}
COND3 -- YES --> CORPS["Display i<br/>i â† i + 1"]
CORPS --> COND3
COND3 -- NO --> FIN2("End")
style DEBUT2 fill:#2ecc71,color:#fff
style COND3 fill:#f39c12,color:#000
style CORPS fill:#3498db,color:#fff
style FIN2 fill:#e74c3c,color:#fff
```

Situation	Recommended loop
Number of iterations known in advance	FOR
Traversing an array of size N	FOR
Waiting for an external condition	WHILE
User input until sentinel value	WHILE

5. Algorithmic Complexity

Complexity measures the number of elementary operations as a function of the input size **n**. We are interested in asymptotic behavior (large n).

```
graph LR
subgraph "Classic complexities"
O1["O(1) â€” Constant<br/>Direct access"]
O2["O(log n) â€” Logarithmic<br/>Binary search"]
O3["O(n) â€” Linear<br/>Simple traversal"]
O4["O(n log n) â€” Quasi-linear<br/>Merge sort"]
O5["O(nÂ²) â€” Quadratic<br/>Nested loops"]
O6["O(2â¿) â€” Exponential<br/>Combinatorial explosion"]
end
O1 --> O2 --> O3 --> O4 --> O5 --> O6
style O1 fill:#2ecc71,color:#fff
style O3 fill:#3498db,color:#fff
style O5 fill:#e67e22,color:#fff
style O6 fill:#e74c3c,color:#fff
```

O(1) â€” Constant: `RETURN T[1]` (1 operation, regardless of n).

O(n) â€” Linear:

```
Algorithm SumArray(T, n)
sum â† 0
FOR i FROM 1 TO n DO
sum â† sum + T[i]
END FOR
RETURN sum // n additions â†’ O(n)
```

O(nÂ²) â€” Quadratic:

```
Algorithm BubbleSort(T, n)
FOR i FROM 1 TO n-1 DO
FOR j FROM 1 TO n-i DO
IF T[j] > T[j+1] THEN
swap T[j] and T[j+1]
END IF
END FOR
END FOR // n(n-1)/2 comparisons â†’ O(nÂ²)

graph TD
subgraph "Comparative growth"
L1["n=10: O(1)=1, O(n)=10, O(nÂ²)=100"]
L2["n=20: O(1)=1, O(n)=20, O(nÂ²)=400"]
L3["n=50: O(1)=1, O(n)=50, O(nÂ²)=2500"]
L4["n=100: O(1)=1, O(n)=100, O(nÂ²)=10000"]
end
style L1 fill:#2ecc71,color:#fff
style L2 fill:#3498db,color:#fff
style L3 fill:#e67e22,color:#fff
style L4 fill:#e74c3c,color:#fff
```

Rule: O(n) is usable for large data, O(nÂ²) is borderline, O(2â¿) is impractical.

6. Manual Execution Trace

The trace allows following variables step by step to understand and verify an algorithm.

Algorithm â€” Factorial of N:

```
Algorithm Factorial(N)
result â† 1; i â† 1
WHILE i â‰¤ N DO
result â† result Ã— i
i â† i + 1
END WHILE
RETURN result
```

Trace for N = 4: (i, result): (1, 1) â†’ (2, 2) â†’ (3, 6) â†’ (4, 24) â†’ (5, FALSE).

Result: Factorial(4) = 24

MCQ Self-Assessment

1. **What is the complexity of a sequential search in an array of size n?**

a) O(1) b) O(n) âœ“ c) O(nÂ²)

2. **Which loop should be used when the number of iterations is known in advance?**

a) WHILE b) FOR âœ“ c) REPEAT

3. **What is the result of Euclid's algorithm for GCD(12, 8)?**

a) 4 âœ“ b) 2 c) 6

4. **Which characteristic guarantees that an algorithm does not loop infinitely?**

a) Unambiguity b) Finiteness âœ“ c) Effectiveness

5. **How many operations does a bubble sort perform for n = 10?**

a) 10 b) Approximately 45 âœ“ c) 100

Conclusion

Algorithmics is the science of problem-solving through step-by-step procedures. Variables, conditions, and loops are the fundamental building blocks. Complexity (O(1), O(n), O(nÂ²)) allows comparing algorithms. The execution trace is an essential pedagogical tool for debugging.

Next session: Data structures â€" arrays, LIFO stacks, FIFO queues, linked lists.

Pedagogical Note ENS

Teaching transposition: In SNT Seconde and NSI PremiÃ¨re:

• **Unplugged activity:** Students follow an algorithm on graph paper serving as memory

• **Cooking recipe:** Ingredients = variables, steps = instructions, repetitions = loops

• **Card sorting:** Have students sort numbered cards following bubble sort or selection sort

• **Complexity:** Count the number of comparisons for different deck sizes

Link with languages: Algorithmics can be compared to grammatical analysis (a sentence breaks down into subjects/verbs/complements like instructions) and to writing an essay outline (sequential structure with conditions).

Trainer advice: Algorithmics is the foundation of all computational thinking. Emphasize the design phase (pseudocode, flowchart) before coding. Use interactive quizzes (Kahoot!, Plickers) to check understanding of concepts (variables, conditions, loops) in a fun way. Propose programming challenges on platforms like France-ioi or CodinGame.

Extension: Students can explore more advanced algorithms (recursion, divide-and-conquer, greedy) and their application in NSI Terminale. Propose a complementary activity: implement merge sort in Python and experimentally compare its complexity with bubble sort (visualization with matplotlib).

Bibliography

• Cormen, T. et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

• Dowek, G. et al. (2019). *SNT Seconde* and *NSI PremiÃ¨re*. Hachette.

• Delannoy, C. (2020). *Algorithmique et programmation en Python*. Eyrolles.

• Biernat, J. et al. (2020). *Algorithmique â€" cours et exercices corrigÃ©s*. Dunod.

• Al-KhwÃ¢rizmÃ® (9th century). *Al-Jabr wa al-MuqÄ□bala*.

• Sedgewick, R. & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_06.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 06 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

At the end of this session, the student is able to:

• Define a data structure and explain its algorithmic role

• Distinguish between direct access (O(1)) and sequential access (O(n)) in an array

• Implement push, pop, peek operations for a stack (LIFO) and enqueue, dequeue for a queue (FIFO)

• Model a linked list with nodes and pointers

• Choose the appropriate structure (array, stack, queue, linked list) based on the problem

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. Concept of a data structure â€” 5 min
2. Arrays â€” 15 min
3. LIFO Stacks â€” 20 min
4. FIFO Queues â€” 20 min
5. Linked lists â€” 15 min
6. Array vs linked list comparison â€” 10 min
7. MCQ and conclusion â€” 5 min

1. Concept of a data structure *(5 min)*

A **data structure** is a logical organization of data in memory enabling efficient operations (addition, deletion, search, traversal).

Selection criteria: access type (direct/sequential), dynamism (fixed/variable size), type of frequent operations.

2. Arrays *(15 min)*

An **array** is a contiguous memory area containing elements of the same type, each identified by an index.

```
// Pseudocode: declaration and access
notes: ARRAY[10] OF INTEGERS
notes[0] â†□ 15
notes[1] â†□ 12
notes[2] â†□ 18

// Traversal and sum
sum â†□ 0
FOR i FROM 0 TO 9 DO
sum â†□ sum + notes[i]
END FOR
```

Fundamental complexities:

• **Index access: O(1)** â€" direct memory address calculation: `addr = base + i Ã— element_size`

• **Sequential traversal: O(n)** â€" each element must be visited

• **Insertion in the middle: O(n)** â€" shifting subsequent elements

Properties: fixed size (static), contiguous in memory, instant random access.

Pedagogical example (NSI): an array of 30 grades for a class. Calculating the average in O(n). Finding the maximum in O(n).

```
graph LR
subgraph "Array notes[5] in memory"
A["Index 0<br/>15"]
B["Index 1<br/>12"]
C["Index 2<br/>18"]
D["Index 3<br/>10"]
E["Index 4<br/>14"]
end
ACCESS["Access notes[2] â†' O(1)<br/>Address = base + 2Ã—4 bytes"] --> C

style A fill:#3498db,color:#fff
style B fill:#3498db,color:#fff
style C fill:#e74c3c,color:#fff
style D fill:#3498db,color:#fff
style E fill:#3498db,color:#fff
style ACCESS fill:#2ecc71,color:#fff
```

3. Stacks â€" LIFO *(20 min)*

Principle: *Last In, First Out* â€" the last element added is the first one removed.

Analogy: a stack of plates. You place the new plate on top; you always take the one from the top.

```
// Stack operations
STACK stack â†□ EMPTY
```

```
push(stack, 10) // [10]
push(stack, 20) // [10, 20]
push(stack, 30) // [10, 20, 30]
x â†□ pop(stack) // x = 30, stack = [10, 20]
y â†□ pop(stack) // y = 20, stack = [10]
```

Operation	Role	Complexity
push(v)	Add to the top	O(1)
pop()	Remove and return the top	O(1)
peek()	View the top without removing	O(1)
is_empty()	Check if the stack is empty	O(1)

```
graph TB
subgraph "LIFO Stack â€” State after push(10), push(20), push(30)"
S3["30 â†□ TOP (push / pop)"]
S2["20"]
S1["10 â†□ BOTTOM"]
end

subgraph "Operations"
P1["push(40) â†’ [10, 20, 30, 40]"]
P2["pop() â†’ returns 30 â†’ [10, 20]"]
P3["peek() â†’ returns 30 (without removing)"]
end

S3 --> P1
S3 --> P2
S3 --> P3

style S3 fill:#e74c3c,color:#fff
style S2 fill:#f39c12,color:#fff
style S1 fill:#3498db,color:#fff
style P1 fill:#2ecc71,color:#fff
style P2 fill:#2ecc71,color:#fff
style P3 fill:#2ecc71,color:#fff
```

Real-world examples:

• **Ctrl+Z** (undo): each action is pushed, Ctrl+Z pops

• **Browser history**: the "Back" button pops visited URLs

• **Call stack**: the system pushes function calls

NSI Teaching: the call stack explains recursion. Each recursive call pushes a new frame; the return pops it.

4. Queues â€” FIFO *(20 min)*

Principle: *First In, First Out* â€” the first element added is the first one removed.

Analogy: a queue at the cafeteria. The first person in line is the first served.

```
// Queue operations
QUEUE queue â†□ EMPTY

enqueue(queue, 10) // [10]
enqueue(queue, 20) // [10, 20]
```

```
enqueue(queue, 30) // [10, 20, 30]
x â†□ dequeue(queue) // x = 10, queue = [20, 30]
y â†□ dequeue(queue) // y = 20, queue = [30]
```

Operation	Role	Complexity
enqueue(v)	Add to the end (tail)	O(1)
dequeue()	Remove and return the front (head)	O(1)
front()	View the head without removing	O(1)
is_empty()	Check if the queue is empty	O(1)

```
graph LR
subgraph "FIFO Queue"
direction LR
HEAD["HEAD<br/>(dequeue)"]
Q1["10"]
Q2["20"]
Q3["30"]
TAIL["TAIL<br/>(enqueue)"]
end

HEAD --> Q1
Q1 --> Q2
Q2 --> Q3
Q3 --> TAIL

ENTRY["enqueue(40) â†’ [10, 20, 30, 40]"] -.-> TAIL
HEAD -.-> EXIT["dequeue() â†’ 10 â†’ [20, 30]"]

style Q1 fill:#e74c3c,color:#fff
style Q2 fill:#f39c12,color:#fff
style Q3 fill:#3498db,color:#fff
style HEAD fill:#2ecc71,color:#fff
style TAIL fill:#2ecc71,color:#fff
style ENTRY fill:#9b59b6,color:#fff
style EXIT fill:#9b59b6,color:#fff
```

Real-world examples:

• **Print queue**: documents are printed in the order they are sent

• **Keyboard buffer**: keystrokes are processed in typing order

• **Task scheduler**: CPU process queues

SNT Teaching: simulate a waiting queue with students (role play). Each student receives a numbered ticket.

5. Linked Lists *(15 min)*

A **linked list** is a dynamic structure where each element (node) contains a value and a pointer to the next node.

```
// Node structure
Node {
value: INTEGER
next: POINTER_TO_Node
}
```

```
graph LR
subgraph "Singly linked list"
N1["Node 1<br/>value: 10<br/>next: *"]
N2["Node 2<br/>value: 20<br/>next: *"]
N3["Node 3<br/>value: 30<br/>next: null"]
end

HEAD["HEAD"] --> N1
N1 -->|"pointer"| N2
N2 -->|"pointer"| N3
N3 --> END["null (end)"]

style N1 fill:#3498db,color:#fff
style N2 fill:#2ecc71,color:#fff
style N3 fill:#f39c12,color:#fff
style HEAD fill:#e74c3c,color:#fff
style END fill:#95a5a6,color:#fff
```

Characteristics:

• **Dynamic size**: no fixed limit, memory is allocated as needed

• **Insertion at the beginning: O(1)** â€" just change the head pointer

• **Insertion in the middle: O(1)** â€" if you have the pointer to the previous node

• **Index access: O(n)** â€" must traverse from the head

• **Search: O(n)** â€" sequential traversal required

Pseudocode for insertion at the beginning:

```
// Insert a value at the beginning of a linked list
FUNCTION insert_beginning(head, value)
new â†□ NEW_Node(value)
new.next â†□ head
RETURN new // new head
END FUNCTION
```

6. Comparison and choice *(10 min)*

```
graph TB
subgraph "Comparison Array vs Linked List"
subgraph "Array"
T1["âœ" Access O(1)<br/>âœ" Contiguous memory<br/>âœ˜ Beginning insertion O(n)<br/>âœ˜ Fixed
size"]
end
subgraph "Linked List"
L1["âœ" Insertion O(1)<br/>âœ" Dynamic size<br/>âœ˜ Access O(n)<br/>âœ˜ Dispersed memory"]
end
end

style T1 fill:#3498db,color:#fff
style L1 fill:#2ecc71,color:#fff
```

Criterion	Array	Linked List
Index access	O(1) â€" direct	O(n) â€" sequential
Beginning insertion	O(n) â€" shifting	O(1) â€" pointer
End insertion	O(1) â€" if space	O(1) â€" if tail known
Deletion	O(n) â€" shifting	O(1) â€" pointer

Search	O(n)	O(n)
Memory	Fixed, contiguous	Dynamic, dispersed
CPU Cache	Favorable (locality)	Unfavorable

Selection guide:

• **Frequent positional access** â†' Array (e.g., indexed grade list)

• **Frequent insertions/deletions at the beginning or middle** â†' Linked list

• **LIFO operations** â†' Stack (e.g., history, undo)

• **FIFO operations** â†' Queue (e.g., waiting line, buffer)

• **Unknown size in advance** â†' Linked list

MCQ Self-Assessment

1. **What is the complexity of accessing an element in an array by its index?**

a) O(n) b) O(1) âœ“ c) O(nÂ²)

2. **In a LIFO stack, which element is removed with pop()?**

a) The first added b) The last added âœ“ c) The element in the middle

3. **Which analogy best represents a FIFO queue?**

a) A stack of plates b) A waiting line at a counter âœ“ c) A display board

4. **In a linked list, what is the cost of an insertion at the beginning?**

a) O(1) âœ“ b) O(n) c) O(nÂ²)

5. **Which data structure does the browser's "Back" button use?**

a) Queue b) Stack âœ“ c) Linked list

Conclusion *(5 min)*

Data structures are essential algorithmic tools. An array favors fast access (O(1)). A linked list favors dynamic insertion (O(1)). The stack implements "last arrived, first served" (LIFO); the queue implements "first arrived, first served" (FIFO). Choosing the right structure determines the program's efficiency.

Next class: Networks and the Internet â€" understanding how computers communicate.

Pedagogical Note ENS

Teaching transposition for secondary school:

• **SNT Year 10 (age 15-16)**: use unplugged activities for stacks (plates, books) and queues (physical waiting line)

• **NSI Year 11 (age 16-17)**: implement a stack in Python with a list; visualize the recursive call stack

• **Cross-curricular link**: in mathematics, stacks are used to evaluate arithmetic expressions (Reverse Polish Notation)

Trainer advice: have students trace the evolution of the stack and queue on the board step by step. Each student manipulates sticky notes representing nodes.

Extension: Students can go further by discovering advanced data structures: binary search trees, graphs, hash tables. Propose a complementary NSI activity: implement a simple spell checker using a hash table to verify words in an English text.

Bibliography

• Cormen, Leiserson, Rivest & Stein â€" *Introduction to Algorithms* (ch. 10: Elementary Data Structures)

• Aho, Ullman â€" *Data Structures and Algorithms*

• Downey â€" *Think Python* (ch. 17: Stacks and Queues)

• Python documentation: `queue` module (thread-safe implementations)

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_07.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 07 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

At the end of this session, the student is able to:

• Distinguish between LAN, WAN, WLAN and give concrete examples

• Read and interpret an IPv4 address and its subnet mask

• Explain the role of DNS in name resolution

• Describe the client-server model with a Web example

• Differentiate between HTTP, HTTPS, FTP, SMTP, IMAP, POP3 protocols

• Understand the principle of cloud computing and its pedagogical uses

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. Types of networks (LAN, WAN, WLAN) â€” 10 min
2. IP addresses, subnet mask and subnet â€” 15 min
3. DNS: the Internet's directory â€” 15 min
4. Client-server model â€” 15 min
5. Web and email protocols â€” 15 min
6. Cloud computing â€” 10 min
7. MCQ and conclusion â€” 5 min

1. Types of networks *(10 min)*

A **computer network** is a set of interconnected devices that exchange data.

Type	Scope	Example
LAN (Local Area Network)	Building, room	Computer lab network
WAN (Wide Area Network)	Region, country, world	The Internet
WLAN (Wireless LAN)	Local, wireless	School Wi-Fi
MAN (Metropolitan Area Network)	City	Urban university network
PAN (Personal Area Network)	~10 m	Bluetooth, smartphone

```
graph TD
subgraph "Typical network topology"
INTERNET[INTERNET<br/>Worldwide WAN]
ISP[ISP<br/>Internet Service Provider]
ROUT[Router<br/>ADSL / Fiber]

ROUT --> SW1[Switch<br/>Wired network]
ROUT --> AP[Access point<br/>Wi-Fi - WLAN]

SW1 --> PC1[Desktop PC 1]
SW1 --> PC2[Desktop PC 2]
SW1 --> SRV[Educational<br/>Server]

AP --> PHONE[Smartphone]
AP --> TAB[Tablet]
AP --> PC3[Laptop]
end

INTERNET --> ISP
ISP --> ROUT

style INTERNET fill:#e74c3c,color:#fff
style ISP fill:#9b59b6,color:#fff
style ROUT fill:#3498db,color:#fff
style SW1 fill:#2ecc71,color:#fff
style AP fill:#f39c12,color:#fff
style SRV fill:#1abc9c,color:#fff
```

Hardware elements:

• **Router**: interconnects networks, routes packets

• **Switch**: connects machines within the same local network

• **Access point**: Wi-Fi extension of the wired network

• **Ethernet cable (RJ45)**: wired connection up to 1 Gbit/s

Pedagogical example: in a classroom, the local network (LAN) connects PCs via a switch. The router provides access to the Internet (WAN) through the ISP. Wi-Fi (WLAN) allows tablets to connect wirelessly.

2. IP addresses and subnet mask *(15 min)*

An **IP address** is a unique identifier assigned to each device connected to a network.

IPv4 (32 bits)

Format: 4 decimal numbers from 0 to 255, separated by dots.

`192.168.1.10` â€” typical private address of a local network

Private address ranges: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`

IPv6 (128 bits)

Format: 8 hexadecimal groups of 16 bits, separated by `:`.

`2001:0db8:85a3:0000:0000:8a2e:0370:7334`

Subnet mask

The **mask** determines which part of the IP address identifies the network and which part identifies the machine.

```
IP : 192.168.1.10
Mask : 255.255.255.0 (/24)
Network : 192.168.1.0
Machine : 0.0.0.10
Number of possible machines: 254 (2â¸ â^’ 2)
```

```
graph LR
subgraph "Decoding an IPv4 address"
IP["192.168.1.10"]
MSK["Mask: 255.255.255.0<br/>(/24)"]
NET["Network part<br/>192.168.1"]
HOST["Machine part<br/>.10"]
BROAD["Broadcast<br/>192.168.1.255"]
end

IP --> MSK
MSK --> NET
MSK --> HOST
NET --> BROAD

style IP fill:#3498db,color:#fff
style MSK fill:#e74c3c,color:#fff
style NET fill:#2ecc71,color:#fff
style HOST fill:#f39c12,color:#fff
style BROAD fill:#95a5a6,color:#fff
```

Essential commands:

```
ip addr # Linux: view IP configuration
ipconfig # Windows: view IP configuration
ping 8.8.8.8 # Test Internet connectivity
traceroute google.com # Visualize packet path
```

3. DNS â€” Domain Name System *(15 min)*

DNS is the Internet's phonebook: it translates domain names into IP addresses.

```
sequenceDiagram
participant U as User
participant B as Browser (Client)
participant DNS as DNS Server
participant W as Web Server

U->>B: Types "www.google.com"
B->>DNS: "What is the IP of www.google.com ?"
DNS->>DNS: Lookup in its database
```

```
DNS-->>B: "142.250.185.4"
B->>W: Connection to 142.250.185.4:443 (HTTP GET)
W-->>B: HTML / JSON Response
B-->>U: Page displayed
```

DNS hierarchy:

1. **Root** (`.`) â€” 13 root servers worldwide
2. **TLD** (`.com`, `.org`, `.dz`, `.fr`) â€” managed by registries
3. **Domain** (`google.com`, `univ-mostaganem.dz`) â€” purchased by the organization
4. **Subdomain** (`www.google.com`, `moodle.univ-mostaganem.dz`)

Command:

```
nslookup google.com # Query DNS
dig google.com # Detailed query (Linux)
```

Pedagogical application: show students that `www.google.com` and `142.250.185.4` refer to the same server. Explain why names are easier to remember than IP addresses.

4. Client-server model *(15 min)*

The **client-server model** is the fundamental architecture of the Web: a client requests a resource, a server provides it.

```
graph LR
subgraph "Client-Server Model"
C1[Browser<br/>HTTP Client]
C2[Mobile app<br/>API Client]
C3[FTP Client]

S1[Web Server<br/>Apache / Nginx]
S2[FTP Server]
S3[Database]
end

C1 -->|"HTTP GET Request"| S1
C2 -->|"REST API Request"| S1
C3 -->|"FTP Connection"| S2
S1 -->|"SQL Query"| S3

S1 -->|"HTML/JSON Response"| C1
S1 -->|"JSON Response"| C2
S2 -->|"Files"| C3

style C1 fill:#3498db,color:#fff
style C2 fill:#2ecc71,color:#fff
style C3 fill:#f39c12,color:#fff
style S1 fill:#e74c3c,color:#fff
style S2 fill:#9b59b6,color:#fff
style S3 fill:#95a5a6,color:#fff
```

Roles: the **client** initiates the request (browser), the **server** provides the resources (always-on machine).

Example: typing `https://moodle.ens-mostaganem.dz` â†’ browser (client) queries DNS â†’ sends HTTP GET request â†’ server responds with HTML â†’ display.

Variants: classic client-server, peer-to-peer (Torrent), 3-tier (client â†’ app â†’ DB).

5. Communication protocols *(15 min)*

TCP/IP Stack

```
graph TB
subgraph "TCP/IP Stack (4 layers)"
APP["Application Layer<br/>HTTP Â· HTTPS Â· FTP Â· SMTP Â· IMAP Â· POP3 Â· DNS"]
TRANS["Transport Layer<br/>TCP (reliable) Â· UDP (fast)"]
INTER["Internet Layer<br/>IP Â· ICMP (ping)"]
ACCESS["Network Access Layer<br/>Ethernet Â· Wi-Fi Â· ADSL"]
end

style APP fill:#e74c3c,color:#fff
style TRANS fill:#3498db,color:#fff
style INTER fill:#2ecc71,color:#fff
style ACCESS fill:#f39c12,color:#fff
```

Web Protocols

Protocol	Port	Usage	Encrypted
HTTP	80	Web browsing (unsecured)	No
HTTPS	443	Secure web browsing (TLS/SSL)	Yes
FTP	21	File transfer	No
SFTP/FTPS	22/990	Secure file transfer	Yes

HTTPS adds a TLS encryption layer between the client and the server.

• Green padlock in the browser

• Digital certificate issued by a Certificate Authority (CA)

• Prevents data snooping on the network

Email Protocols

• **SMTP** (Simple Mail Transfer Protocol) â€” port 25 or 587: sending emails

• **IMAP** (Internet Message Access Protocol) â€” port 143: receiving and multi-device synchronization (messages remain on the server)

• **POP3** (Post Office Protocol v3) â€” port 110: receiving and local download (messages are deleted from the server)

IMAP vs POP3 comparison:

IMAP	POP3
Messages on the server	Messages downloaded locally
Multi-device synchronization	Single device
Access from anywhere	Offline access
Recommended for professional use	Use on a single computer

For the instructor: use **IMAP** (multi-device sync), **institutional email** for all official communication, and write professional emails (clear subject, salutation, signature).

6. Cloud computing *(10 min)*

Cloud computing is the on-demand availability of computer resources (storage, computing, applications) over the Internet, without direct management of the infrastructure.

Service Models

Model	Example	Teacher use
SaaS (Software as a Service)	Moodle, Google Classroom, Office 365	Learning platform
PaaS (Platform as a Service)	Heroku, PythonAnywhere	Application deployment
IaaS (Infrastructure as a Service)	AWS, Azure, OVH	Virtual machines

Cloud services for the teacher

• **OneDrive / Google Drive**: storage, sharing and backup of teaching documents

• **Moodle**: course repository, assignment submission, forums, online quizzes

• **GitHub / GitLab**: code hosting, version control, NSI projects

• **Overleaf**: collaborative LaTeX editing

• **Padlet / Framapad**: real-time collaboration

Advantages: accessibility (from anywhere), automatic backup, easy sharing, automatic updates.

Disadvantages: dependency on Internet connection, data privacy (GDPR), potential cost.

MCQ Self-Assessment

1. **What does LAN stand for?**

a) Large Area Network b) Local Area Network âœ“ c) Long Access Network

2. **What is the size of an IPv4 address?**

a) 16 bits b) 32 bits âœ“ c) 128 bits

3. **What is the role of DNS?**

a) Encrypt communications b) Translate domain names into IP addresses âœ“ c) Manage passwords

4. **Which protocol guarantees encrypted Web communication?**

a) HTTP b) HTTPS âœ“ c) FTP

5. **Which email protocol is recommended for multi-device synchronization?**

a) POP3 b) SMTP c) IMAP âœ“

Conclusion *(5 min)*

The Internet relies on a hierarchical infrastructure (routers, DNS servers) and standardized protocols (TCP/IP, HTTP, SMTP). The client-server model organizes exchanges. Security (HTTPS) and the cloud are transforming educational practices. As a future teacher, mastering these basics allows you to use digital tools with discernment.

Next class: Computer security and digital ethics.

Pedagogical Note ENS

Teaching transposition for secondary school:

• **SNT Year 10** (theme "Internet"): use `ping` and `traceroute` to visualize the data path. Unplugged activity: students as routers with routing tables.

• **NSI Year 11**: model client-server with Python sockets. TCP/IP architecture in detail.

• **Civics (EMC)**: debate cloud issues (personal data, digital sovereignty).

Trainer advice: simulate a DNS request: one student is the client, another the DNS, another the web server. They exchange envelopes with IP addresses.

Extension: Students can explore application layers (HTTP/2, DNS, DHCP, SMTP) and simulate a client-server dialogue with Python sockets. Propose a complementary activity: create a small web server in Python with Flask and deploy it locally to understand how a web application works.

Bibliography

• Kurose & Ross â€" *Computer Networking: A Top-Down Approach* (8th ed.)

• Tanenbaum & Wetherall â€" *Computer Networks* (6th ed.)

• ANSSI â€" *Network Security Guide* (available at www.ssi.gouv.fr)

• RFC 1034 â€" *Domain Names â€" Concepts and Facilities*

• RFC 2616 â€" *HTTP/1.1*

• Mozilla Documentation: *How the Internet works* (developer.mozilla.org)

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_08.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 08 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

At the end of this session, the student is able to:

• Explain the CIA triad (Confidentiality, Integrity, Availability)

• Create a strong password and understand the principle of MFA

• Identify signs of a phishing attempt

• Apply the 3-2-1 backup rule

• Know the GDPR principles applicable in a school environment

• Adopt responsible digital ethics (cyberbullying, copyright, generative AI)

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. CIA triad (Confidentiality, Integrity, Availability) â€” 15 min
2. Strong passwords and multi-factor authentication â€” 15 min
3. Digital threats (phishing, malware, ransomware) â€” 15 min
4. 3-2-1 backup rule â€” 10 min
5. GDPR and student data protection â€” 15 min
6. Digital ethics for the teacher â€” 15 min
7. MCQ and conclusion â€” 5 min

1. CIA Triad *(15 min)*

Computer security rests on three fundamental pillars, often represented by the CIA triangle (Confidentiality, Integrity, Availability).

```
graph TB
CIA((COMPUTER<br/>SECURITY))

CIA --> CONF[Confidentiality<br/>Data accessible only to<br/>authorized users]
CIA --> INT[Integrity<br/>Data not modified<br/>in an unauthorized way]
CIA --> AVAIL[Availability<br/>Data and services<br/>accessible when needed]

CONF <-->|Dependency| INT
INT <-->|Dependency| AVAIL
AVAIL <-->|Dependency| CONF

style CIA fill:#e74c3c,color:#fff
style CONF fill:#3498db,color:#fff
style INT fill:#2ecc71,color:#fff
style AVAIL fill:#f39c12,color:#fff
```

Confidentiality: ensuring that only authorized persons can access the information.

Mechanism	Concrete example
Encryption	HTTPS, encrypted hard drive (BitLocker)
Access control	Password, RFID badge
Authentication	MFA, biometrics

Integrity: ensuring that data has not been altered.

Mechanism	Concrete example
Checksum (hash)	MD5, SHA-256 (downloads)
Digital signature	Signed email, SSL certificates
Logging	Audit trail, modification history

Availability: ensuring access to data and services when needed.

Mechanism	Concrete example
Redundancy	Mirror servers, RAID
Backup	3-2-1 rule
Recovery plan	DRP (Disaster Recovery Plan)

Pedagogical example: an online grade book. **Confidentiality** ensures only the teacher and administration can access it. **Integrity** prevents a student from modifying their grade. **Availability** ensures it can be accessed on the day of the parent-teacher conference.

2. Passwords and authentication *(15 min)*

Characteristics of a strong password

Criterion	Weak

Length	6-8 characters
Complexity	lowercase only
Dictionary	password, 123456, qwerty
Personal	Date of birth, first name

Examples:

• `sunshine2025` â†' WEAK (dictionary word + simple year)

• `ENS.M0st@g4nem_2026!` â†' STRONG (â‰¥12 chars, mixed types, special characters)

Multi-Factor Authentication (MFA)

Combination of several **factors** of authentication:

```
FACTOR 1: Something I know â†' Password, PIN code
+
FACTOR 2: Something I have â†' SMS, app (Google Auth), USB key
+
FACTOR 3: Something I am â†' Fingerprint, facial recognition
```

MFA in schools: enable two-factor authentication on Moodle, institutional email, and academic services.

Password Manager

• **Principle:** a single master password to unlock a digital vault

• **Tools:** Bitwarden (free, open source), KeePass, Apple Passwords

• **Advantage:** generates unique and complex passwords for each service

3. Digital threats *(15 min)*

Phishing

Social engineering technique where the attacker impersonates a legitimate entity to steal credentials.

```
graph TB
subgraph "Phishing attack cycle"
E[Step 1<br/>Creation of a<br/>fraudulent email] -->|"Sender:<br/>support@g00gle.com"|
D[Step 2<br/>Mass sending<br/>to victims]
D -->|"Subject:<br/>'Account blocked'"| C[Step 3<br/>Victim clicks<br/>on the link]
C -->|"Link:<br/>http://fake.com"| S[Step 4<br/>Entering credentials<br/>on trapped page]
S -->|"Password theft"| F[Step 5<br/>Fraudulent use<br/>of the account]
end

style E fill:#e74c3c,color:#fff
style D fill:#f39c12,color:#fff
style C fill:#3498db,color:#fff
style S fill:#9b59b6,color:#fff
style F fill:#e74c3c,color:#fff
```

Warning signs:

1. Suspicious sender address (e.g., `support@g00gle.com` with two zeros)
2. Spelling and grammar mistakes
3. Urgent message ("your account will be closed within 24h")
4. Fraudulent link (hover over the link without clicking to see the real URL)

5. Unexpected attachment (`.zip`, `.exe`, `.docm`)

Malware

Type	Description	Propagation
Virus	Attaches to a legitimate file	Email attachments, downloads
Worm	Self-propagates across the network	Email, network vulnerabilities
Trojan Horse	Poses as legitimate software	Trapped download
Spyware	Spies on user activity	Websites, free software
Ransomware	Encrypts files and demands ransom	Phishing, USB drives

Ransomware in detail:

1. Infection via malicious attachment or USB drive
2. File encryption (documents, photos, databases)
3. Display of a ransom demand (often in Bitcoin)
4. The victim loses data if they don't pay (and even if they do)

Antivirus protection: Windows Defender (built into Windows), free solutions (ClamAV), regular audits.

4. 3-2-1 Backup Rule *(10 min)*

The **3-2-1 rule** is the minimum backup strategy recommended by security experts.

```
graph LR
subgraph "3-2-1 Backup Strategy"
ORIG["Original copy<br/>PC / Server"]
LOCAL["Local copy<br/>External drive / NAS"]
REMOTE["Remote copy<br/>Cloud / Different site"]
end

ORIG -->|"2. Local backup"| LOCAL
ORIG -->|"3. Remote backup"| REMOTE

NOTE["3 copies Â· 2 different media Â· 1 off-site copy"]

style ORIG fill:#e74c3c,color:#fff
style LOCAL fill:#3498db,color:#fff
style REMOTE fill:#2ecc71,color:#fff
style NOTE fill:#f39c12,color:#fff,stroke:#333,stroke-dasharray: 5 5
```

Practical application for the teacher:

```
Original data (laptop)
â”,
â”œâ”€€ Copy 1: USB external hard drive (same location)
â”,
â””â”€â”€€ Copy 2: Institutional OneDrive cloud (off-site)
```

Backup schedule:

• **Daily**: course documents modified during the day

• **Weekly**: full backup of teaching folders

• **Monthly**: archived backup (beginning and end of month)

Pitfalls to avoid:

• Not testing backups (an untested backup doesn't exist)

• Keeping only one copy (in the same place)

• Storing data on an unencrypted USB drive

5. GDPR and data protection *(15 min)*

GDPR (General Data Protection Regulation) â€" European Regulation 2016/679, applicable since May 2018.

Fundamental Principles

Principle	Application in schools
Minimization	Collect only strictly necessary data (name, class, no medical data without necessity)
Transparency	Inform students and their parents about data usage
Consent	Obtain permission for photos, publications
Right of access	Students or parents can request to see their data
Right to erasure	Delete data when no longer needed (end of year)
Security	Protect student lists, grades and comments

Particularly sensitive data to protect

```
Student list (last name, first name, date of birth)
Grades, comments, report cards
Parent address, phone number
Student photos
Special accommodations (IEP, 504 plan, medical notifications)
```

Best practices for the teacher

• Use exclusively **institutional tools** (Moodle, university email, academic OneDrive)

• **Do not store** student data on personal services (personal Google Drive, Dropbox)

• **Anonymize** data in publications and research

• **Retention period** limited to the current school year

• **Encrypt** sensitive files (Word/Excel with password, encrypted hard drive)

Reference: ICO (UK) / CNIL (France) â€" *Data protection guide for the education sector*

6. Digital ethics for the teacher *(15 min)*

Cyberbullying

• Forms: insults, rumors, identity theft, sharing intimate photos

• Statistics: 1 in 5 young people reports having been a victim (source: UNICEF)

• **Teacher's role:** detect, report (3018 in France, cybersignalement platform), support

• **Prevention:** civics sessions, school digital charter

Copyright and licenses

• **Images:** use royalty-free sources (Unsplash, Pixabay, Wikimedia Commons)

• **Creative Commons (CC) licenses:** CC0 (public domain), CC BY (attribution required), CC BY-NC (non-commercial use only)

• **Source citation:** mandatory in all teaching documents

• **Texts:** public domain (law: 70 years after author's death) or publisher agreement

Generative Artificial Intelligence

Issues for teaching (2024-2026):

Opportunity	Risk
Course writing assistant	Student plagiarism
Custom exercise generation	Circumventing assessments
Automatic translation of instructions	Hallucinations and inaccuracies
Grading assistance	Bias and discrimination

Recommendations:

• **Define an explicit policy** on AI use in the classroom

• **Train students** in critical use of AI

• **Adapt assessments**: more oral work, analysis, in-class production

• **Cite systematically** when AI is used to produce a document

Digital footprint

• Everything published online leaves a trace (posts, photos, comments)

• **Advice for future teachers:** configure privacy settings on personal accounts, separate professional and personal life, never publish anything that could harm your professional image

MCQ Self-Assessment

1. **What does the "C" stand for in the CIA triad?**

a) Connection b) Confidentiality âœ“ c) Certification

2. **What is the minimum recommended length for a strong password?**

a) 8 characters b) 12 characters âœ“ c) 6 characters

3. **What is ransomware?**

a) An antivirus software b) A program that encrypts files and demands a ransom âœ“ c) A firewall

4. **What is the 3-2-1 rule?**

a) 3 passwords, 2 emails, 1 browser b) 3 copies, 2 media, 1 off-site copy âœ“ c) 3 users, 2 servers, 1 backup

5. **According to GDPR, student data must be:**

a) Kept indefinitely b) Deleted after the school year âœ“ c) Published online

Conclusion *(5 min)*

Computer security is everyone's business. Knowing the CIA triad, creating strong passwords, recognizing phishing, applying the 3-2-1 rule, and respecting GDPR are essential skills for any 21st-century teacher. Digital ethics (cyberbullying, copyright, AI) must be integrated into your daily teaching practice.

Next class: Introduction to Python for teaching.

Pedagogical Note ENS

Teaching transposition for secondary school:

• **SNT Year 10** (theme "Personal data"): analyze social network privacy settings, create a class digital charter

• **NSI Year 11**: symmetric/asymmetric encryption, digital signature, SSL certificates

• **Civics (EMC)**: debate on cyberbullying, freedom of expression online, deepfakes

Recommended activities:

1. Collectively analyze 3 emails (1 legitimate, 1 obvious phishing, 1 sophisticated phishing)

2. Simulate a ransomware attack on a dummy folder

3. Calculate the time needed to crack a 6 vs 12 character password

4. Write a simplified GDPR charter for the class

Trainer advice: Approach cybersecurity with concrete examples drawn from school life (Pronote password, phishing on academic email, personal data on social networks). Emphasize the teacher's role as the first line of defense against cyber threats in schools. Use serious games like "CyberEngage" or "HackApp" to raise awareness without causing anxiety.

Extension: Students can explore modern cryptography (RSA, AES) and encrypted messaging (Signal, ProtonMail). Propose a complementary activity: implement a Caesar cipher in Python, then a substitution cipher, and finally discuss the limitations of these methods against frequency analysis.

Bibliography

• Stallings & Brown â€" *Computer Security: Principles and Practice* (4th ed.)

• ANSSI â€" *Guide for selecting encryption algorithms* (ssi.gouv.fr)

• ICO â€" *Data protection in education* (ico.org.uk)

• GDPR â€" Regulation (EU) 2016/679 of the European Parliament (eur-lex.europa.eu)

• UK Intellectual Property Office â€" *Copyright guide for teachers*

• MIT â€" *Cybersecurity for Everyone* (online course)

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_09.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 09 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

At the end of this session, the student is able to:

• Justify the choice of Python for teaching programming

• Install and configure a Python environment (IDLE, VS Code, Thonny)

• Declare and use basic types (int, float, str, bool)

• Structure a program with conditions (if/elif/else) and loops (for/while)

• Define and call functions with parameters and return values

• Read and write a text file from a Python script

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. Why Python for teaching â€” 10 min
2. Development environments â€” 10 min
3. Variables and types â€” 15 min
4. Conditions (if/elif/else) â€” 15 min
5. Loops (for, while) â€” 15 min
6. Functions (def, return, parameters) â€” 15 min
7. Input/output and files â€” 10 min

8. MCQ and conclusion â€” 5 min

1. Why Python for teaching? *(10 min)*

Python has become the most widely used programming language in education and research.

```
graph LR
subgraph "Algorithmic pseudocode"
PC["IF grade >= 10<br/> DISPLAY 'Passed'<br/>ELSE<br/> DISPLAY 'Failed'"]
end
subgraph "Python code"
PY["if grade >= 10:<br/> print('Passed')<br/>else:<br/> print('Failed')"]
end
subgraph "Execution"
EX["Input: 15<br/>Output: Passed"]
end
PC -->|"Natural translation"| PY
PY -->|"Interpretation"| EX
style PC fill:#f39c12,color:#fff
style PY fill:#3498db,color:#fff
style EX fill:#2ecc71,color:#fff
```

Pedagogical strengths: readable syntax (close to pseudocode), dynamic typing, free/open source, rich libraries, active educational community. Python is the official language of the NSI program in France.

Uses in school contexts:

• **Mathematics:** calculations, simulation, plotting (matplotlib)

• **Languages:** text analysis, vocabulary quizzes

• **Science:** data processing, graphs

• **SNT/NSI:** official program â€” reference language

2. Development environments *(10 min)*

Tool	Type	Target audience	Advantage
IDLE	Built-in IDE	Beginners	Installed with Python
Thonny	Educational IDE	Middle/High school	Visual debugger, ideal for discovery
VS Code	Professional editor	High school/University	Extensions, integrated terminal
Replit	Online	All	No installation required
Google Colab	Web Notebook	Research/NSI	Shared tutorials

Recommendation: Thonny for beginners, then VS Code for projects.

```
# Check Python
python --version
# Run a script
echo 'print("Hello ENS !")' > hello.py && python hello.py
```

3. Variables and types *(15 min)*

In Python, variables are labels pointing to objects. No explicit type declaration.

```
age = 20 # int
pi = 3.14159 # float
name = "Madani" # str
is_teacher = True # bool

print(type(age)) # <class 'int'>
print(f"{name} is {age} years old") # f-string (Python 3.6+)

# Type conversion
age_str = "20"
age_int = int(age_str) # "20" â†’ 20
grade = float(input("Grade: ")) # Input â†’ float
```

Pseudocode â†” Python parallel:

Concept	Pseudocode	Python
Assignment	age â† 20	age = 20
Display	DISPLAY age	print(age)
Input	READ age	input()
Comment	// text	# text
Comparison	age == 18	age == 18

Pitfall: = is assignment, == is comparison.

4. Conditions *(15 min)*

```
graph TD
START(["Start"]) --> INPUT[Input grade]
INPUT --> COND{"grade >= 10 ?"}
COND -->|Yes| PASSED[Display 'Passed']
COND -->|No| FAILED[Display 'Failed']
PASSED --> END(["End"])
FAILED --> END
style START fill:#2ecc71,color:#fff
style END fill:#e74c3c,color:#fff
style COND fill:#f39c12,color:#fff
```

```
grade = float(input("Grade out of 20: "))
if grade >= 16:
print("Excellent")
elif grade >= 14:
print("Good")
elif grade >= 12:
print("Fairly good")
elif grade >= 10:
print("Passable")
else:
print("Failed")
```

Operators: ==, !=, <, >, <=, >=, and, or, not

Golden rules:

- Don't forget the : after `if`, `elif`, `else`
- Consistent indentation (4 spaces)
- `float(input())` for decimal numbers

5. Loops *(15 min)*

`for` loop â€” sequence traversal

```
for i in range(5):
print(f"Iteration {i}")

for i in range(1, 10, 2):
print(i) # 1, 3, 5, 7, 9

names = ["Ali", "Sara", "Mohamed"]
for name in names:
print(f"Hello {name}")

for i, name in enumerate(names):
print(f"{i+1}. {name}")
```

`while` loop â€” conditional repetition

```
response = ""
while response != "yes" and response != "no":
response = input("Continue? (yes/no): ").lower()

counter = 0
while counter < 5:
print(f"Counter: {counter}")
counter += 1

graph LR
subgraph "for loop"
F1["for i in range(5):"]
F2["print(i)"]
F3["i: 0 â†’ 1 â†’ 2 â†’ 3 â†’ 4"]
end
subgraph "while loop"
W1["while condition:"]
W2["statement"]
W3["Condition checked<br/>at each iteration"]
end
F1 --> F2 --> F3
W1 --> W2 --> W3 --> W1
style F1 fill:#3498db,color:#fff
style W1 fill:#e74c3c,color:#fff
```

When to use which? `for` when the number of iterations is known (sequence traversal). `while` when waiting for a condition.

6. Functions *(15 min)*

```
def calculate_average(grades):
"""Calculates the average of a list of grades."""
return sum(grades) / len(grades)

def greet(name, message="Welcome"):
print(f"{message}, {name}!")

grades = [15, 12, 18, 10, 14]
avg = calculate_average(grades)
print(f"Average: {avg:.2f}")

greet("Ali") # Welcome, Ali!
greet("Sara", "Hello") # Hello, Sara!
```

```
def min_max_grades(grades):
return min(grades), max(grades) # Returns a tuple

mini, maxi = min_max_grades(grades)
print(f"Min: {mini}, Max: {maxi}")
```

Why functions? Reusability, readability, testability, modularity.

Best practices: verb names, docstring between """, one return per function.

7. Input/output and files *(10 min)*

```
# User input
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# File writing
with open("grades.txt", "w", encoding="utf-8") as f:
f.write("Name,Grade\n")
f.write("Ali,15\n")
f.write("Sara,18\n")
# 'with' closes automatically

# File reading
with open("grades.txt", "r", encoding="utf-8") as f:
for line in f:
print(line.strip())

# Modes: "r" read, "w" write (overwrites), "a" append

# Simple CSV processing
with open("grades.txt", "r") as f:
f.readline() # Skip header
total = 0
n = 0
for line in f:
name, grade = line.strip().split(",")
total += float(grade)
n += 1
print(f"Average: {total/n:.2f}")
```

Pedagogical context: the CSV format is ubiquitous in education (Moodle export, Excel, report cards).

8. Mini-project *(5 min)*

English/French vocabulary quiz â€” combining all concepts:

```
questions = {
"cat": "chat", "dog": "chien", "house": "maison",
"book": "livre", "school": "Ã©cole", "teacher": "professeur"
}

def ask_question(word_en, word_fr):
answer = input(f"Translate '{word_en}': ").strip().lower()
if answer == word_fr:
print("âœ“ Correct!")
return True
print(f"âœ— Wrong. Answer: {word_fr}")
return False

score = 0
```

```
for en, fr in questions.items():
if ask_question(en, fr):
score += 1
print(f"Score: {score}/{len(questions)}")
```

MCQ Self-Assessment

1. **Which function displays text in the console in Python?**

a) `input()` b) `print()` â€“ c) `write()`

2. **What type does `input()` return?**

a) `int` b) `float` c) `str` â€“

3. **Which keyword defines a function?**

a) `function` b) `def` â€“ c) `define`

4. **Which loop is ideal for traversing a list?**

a) `while` b) `for` â€“ c) `repeat`

5. **Which opening mode allows writing (overwrites)?**

a) `"r"` b) `"w"` â€“ c) `"a"`

Conclusion *(5 min)*

Python is ideal for teaching: clear syntax, universal concepts (variables, conditions, loops, functions), concrete applications (file processing, quizzes). CSV manipulation prepares future teachers to handle educational data.

Next class: Python project â€" revision card generator and average calculator.

Pedagogical Note ENS

Teaching transposition for secondary school:

Level	Theme	Recommended activity
SNT Year 10	Programming	Use Thonny for first steps. Write a script that displays "Hello class!" and modify it co
NSI Year 11	Types and functions	Implement a average(grades) function that calculates
NSI Year 12	Recursion and files	Read a CSV grade file, calculate statistics, export a report card.
Mathematics	Simulation	Write a function that simulates a die roll (random), repeat N times, count frequencie
Cross-curricular	Vocabulary quiz	The mini-project quiz (section 8) can be adapted in language class: the student prog

Progressive pedagogical approach:

1. **Read**: analyze an existing script, identify blocks (variables, conditions, loops)
2. **Modify**: change values, add a condition, extend a loop
3. **Create**: write a script meeting a simple specification
4. **Evaluate**: test with different datasets, detect and fix errors

Trainer advice:

• Use **Thonny** for discovery (step-by-step visible debugger), then **VS Code** for projects

• Prioritize **contextual exercises** (average calculator, quiz, converter) rather than decontextualized exercises

• **Systematically** show the pseudocode â†" Python parallel to anchor concepts seen in the algorithms course (session 05)

• Prepare **help sheets** (Python cheatsheet) that students keep for subsequent sessions

• **Avoid:** using portable Thonny or Replit for the first sessions (prefer a local installation)

Extension: Students can explore Python libraries for teaching: `turtle` (graphics), `pygame` (games), `matplotlib` (visualization). Propose a complementary activity: create a small French vocabulary exercise with random flashcards using `tkinter` or an interactive command-line quiz with timer (`time`). These activities are directly transferable to language or history-geography classes.

Bibliography

• Downey â€" *Think Python* (2nd ed., O'Reilly)

• Python 3 Documentation: docs.python.org

• *Python for Teaching* (Eduscol)

Templates : `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/PG-fr.docx` | `C:/Users/madan/Desktop/Supports de Cours 250626/Templatte/Presentation_BELACEL_V3.pptx`

PDF : `Cours_Informatique_ENS_1Ã¨re_annÃ©e_AN_10.pdf`

Computer Science Course â€” ENS Year 1 â€” Session 10 (EN)

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS â€” Year 1 â€” English Department

Duration: 1h30

Academic Year: 2025-2026

Specific Learning Objectives

At the end of this session, the student is able to:

• Structure a Python project into modules (main.py, functions.py, data.csv)

• Handle runtime errors with try/except

• Read and write CSV files with the csv module

• Develop a revision card generator from a CSV

• Develop an average calculator with CSV export

• Present their project according to the defense grading rubric

Algerian Regulatory Context

This course falls within the Algerian regulatory framework for teacher training:

• **Ministerial decree establishing ENS (Bachelor's/Master's cycle)** â€” defines the institutional framework of the Ã‰coles Normales SupÃ©rieures and their mission of teacher training.

• **National Computer Science curriculum for Algerian ENS** â€” sets the objectives, content and hourly volumes of the Computer Science module in initial teacher training.

• **Decree 712 setting the ENS competition syllabus** â€” determines the tests and programs for ENS entrance competitions, consistent with the taught content.

• **Algerian digital competency framework** â€” national framework (under development) defining the digital competencies expected of teachers.

• **Benchmark of 10 Algerian universities** â€” comparative study of computer science teaching practices, consulted to align this course with national standards.

Course Plan

1. Python recap â€” pseudocode â†” Python parallel â€” 10 min
2. Structure of a Python project â€” 10 min
3. Error handling (try/except) â€” 15 min
4. csv module (read/write) â€” 10 min
5. Project: Revision card generator â€” 20 min
6. Project: Average calculator with CSV export â€” 15 min
7. Defense tips and grading rubric â€” 10 min

1. Python Recap *(10 min)*

Concept	Pseudocode	Python
Variable	age â†□ 20	age = 20
Condition	IF age >= 18	if age >= 18:
For loop	FOR i FROM 0 TO 4	for i in range(5):
While loop	WHILE x > 0	while x > 0:
Function	FUNCTION f(x):	def f(x):
Return	RETURN x	return x
Array	t[1] (index 1)	t[0] (index 0)
Display	DISPLAY x	print(x)
Input	READ x	x = input()

Warning: Python lists start at index 0, unlike pseudocode which often starts at 1.

2. Structure of a Python project *(10 min)*

```
graph TD
ROOT[teaching_project/] --> MAIN[main.py]
ROOT --> FUNC[functions.py]
ROOT --> CSV[data.csv]
MAIN -->|imports| FUNC
FUNC -->|reads/writes| CSV
style ROOT fill:#34495e,color:#fff
style MAIN fill:#e74c3c,color:#fff
style FUNC fill:#3498db,color:#fff
style CSV fill:#2ecc71,color:#fff
```

```
teaching_project/
â”œâ”€â”€ main.py # Entry point
â”œâ”€â”€ functions.py # Business functions
â”œâ”€â”€ data.csv # Data
â””â”€â”€ README.txt # Documentation
```

```
# functions.py
def calculate_average(grades): return sum(grades)/len(grades)

def grade_label(m): return "A" if m>=16 else "B" if m>=14 else "C" if m>=12 else "D" if m>=10
else "F"
```

3. Error handling â€” try/except *(15 min)*

```
graph TD
START([Start]) --> TRY["try: grade = float(input())"]
TRY --> CHECK{"0 <= grade <= 20 ?"}
CHECK -->|Yes| OK[Display]
CHECK -->|No| RAISE["raise ValueError"]
RAISE --> EXCEPT["except ValueError:"]
TRY -.->|error| EXCEPT
EXCEPT --> ERR[Message]
OK --> END; ERR --> END
style TRY fill:#3498db; style EXCEPT fill:#e74c3c
```

```
while True:
try:
grade = float(input("Grade (0-20): "))
if 0 <= grade <= 20:
break
print("Grade must be between 0 and 20.")
except ValueError:
print("Please enter a number.")

try:
with open("grades.csv", "r") as f:
data = f.read()
except FileNotFoundError:
print("File not found.")
except PermissionError:
print("Permission denied.")
```

Rules: specify the exception type (`ValueError`, `FileNotFoundError`), provide a clear message, never use bare `except:`.

4. csv module *(10 min)*

```
import csv

# Reading (list of lists)
with open("grades.csv", "r", encoding="utf-8") as f:
reader = csv.reader(f)
next(reader) # Skip header
for row in reader:
name, grade1, grade2 = row
print(f"{name}: {grade1}, {grade2}")

# Reading (dictionaries)
with open("grades.csv", "r", encoding="utf-8") as f:
reader = csv.DictReader(f)
for row in reader:
print(row["name"], row["grade1"])

# Writing
with open("results.csv", "w", newline="", encoding="utf-8") as f:
writer = csv.writer(f)
writer.writerow(["Name", "Average", "Grade"])
writer.writerow(["Ali", 15.5, "Good"])
writer.writerow(["Sara", 18.0, "Excellent"])

# Writing with DictWriter
with open("results.csv", "w", newline="", encoding="utf-8") as f:
columns = ["Name", "Average", "Grade"]
writer = csv.DictWriter(f, fieldnames=columns)
writer.writeheader()
writer.writerow({"Name": "Ali", "Average": 15.5, "Grade": "Good"})
```

Important: always use `newline=""` when writing (avoids double line spacing on Windows).

5. Project â€” Revision Card Generator *(20 min)*

Step 1: `revision.csv`

```
Concept,Definition
Variable,Box that stores a value in memory
Function,Reusable block of code
```

```
Loop,Repeats a block of instructions
Condition,Executes based on a boolean expression
Stack,LIFO structure (last added = first removed)
Queue,FIFO structure (first added = first removed)
DNS,Translates domain names into IP addresses
```

Steps 2-3: `functions.py` + `main.py`

```
import csv, random

def load_cards(f):
try:
with open(f, encoding="utf-8") as fh:
return list(csv.DictReader(fh))
except FileNotFoundError:
print(f"File not found.")
return []

def run_quiz(cs):
if not cs: return
random.shuffle(cs); score = 0
for c in cs:
print(f"\nConcept: {c['Concept']}")
input("Press Enter..."); print(f"Definition: {c['Definition']}")
if input("Correct? (y/n): ")=="y": score+=1
print(f"Score: {score}/{len(cs)}")

if __name__ == "__main__":
run_quiz(load_cards("revision.csv"))
```

6. Project â€” Average Calculator *(15 min)*

```
import csv

def input_grades():
students = []
print("\n--- GRADE ENTRY ---")
while True:
name = input("\nName (or 'done'): ").strip()
if name.lower() == "done": break
grades = []
for i in range(1, 4):
while True:
try:
n = float(input(f" Grade {i} (0-20): "))
if 0 <= n <= 20:
grades.append(n); break
print("Invalid grade.")
except ValueError:
print("Enter a number.")
avg = round(sum(grades)/len(grades), 2)
students.append({"name": name, "grades": grades, "average": avg})
print(f"â†’ {name}: {avg:.2f}")
return students

def stats(students):
if not students: return
averages = [s["average"] for s in students]
print(f"\nCount: {len(students)} | Class avg: {sum(averages)/len(averages):.2f}")
print(f"Max: {max(averages):.2f} | Min: {min(averages):.2f}")

def export(students, file="results.csv"):
```

```
with open(file, "w", newline="", encoding="utf-8") as f:
cols = ["Name", "Grade1", "Grade2", "Grade3", "Average", "GradeLabel"]
w = csv.DictWriter(f, fieldnames=cols)
w.writeheader()
for s in students:
avg = s["average"]
label = "A" if avg >= 16 else "B" if avg >= 14 else "C" if avg >= 12 else "D" if avg >= 10
else "F"
w.writerow({"Name": s["name"], "Grade1": s["grades"][0],
"Grade2": s["grades"][1], "Grade3": s["grades"][2],
"Average": f"{avg:.2f}", "GradeLabel": label})
print(f"Exported to {file}")

if __name__ == "__main__":
s = input_grades()
if s: stats(s); export(s)
```

7. Defense Tips *(10 min)*

Criterion	Weight	Expectations
Functionality	30%	Error-free execution
Code structure	20%	Separate files, docstrings
Error handling	15%	try/except
CSV data	10%	Correct import/export
README.txt	10%	Purpose, usage
Oral presentation	15%	5 min demo + questions

Defense structure: presentation (30s) â† demo (2min30) â† code (1min) â† improvements (30s) â† questions (30s).

Project	CSV	Features
Vocabulary quiz	words.csv	Random draw, score
Multiplication tables	difficulties.csv	Random exercises
Grade manager	students.csv	Averages, report export
Flashcards	concepts.csv	Learning mode

MCQ Self-Assessment

1. **How to import functions from a file `functions.py`?**

a) `include functions` b) `from functions import *` âœ“ c) `using functions`

2. **Which exception for a file not found?**

a) `ValueError` b) `FileNotFoundError` âœ“ c) `TypeError`

3. **Which parameter avoids double line spacing in CSV on Windows?**

a) `encoding="utf-8"` b) `newline=""` âœ“ c) `mode="wb"`

4. **Which csv method returns rows as dictionaries?**

a) `csv.reader()` b) `csv.DictReader()` âœ“ c) `csv.reader_dict()`

5. **Weight of Functionality in the rubric?**

a) 20% b) 30% âœ“ c) 15%

Conclusion *(5 min)*

This last course gave you the tools for a complete Python project: modules, error handling, CSV, and defense. You are able to create concrete pedagogical tools (quizzes, exercise generators, grade calculators) for your future classes.

Congratulations!

Pedagogical Note ENS

Module Review

The final Python project validates all the module's competencies (C1â€“C8). The approach followed (specifications â†' design â†' implementation â†' testing â†' defense) is exactly that of the NSI Baccalaureate in France.

Teaching transposition for secondary school

Competency	Transposable activity for high school
Project structures	Splitting into main.py / functions.py / <font fa
Error handling	try/except with explicit messages: required in NSI Year 12 to validate the "
CSV files	The CSV format (Moodle export, Excel, gradebook) is the most common data exchange format in schoo
Pedagogical quiz	Interactive program usable in all subjects: languages, history, science
Average calculator	Automating grade tracking: a concrete tool the future teacher can deploy in their classroom

Recommended classroom activities

1. **Existing project analysis**: Distribute a complete Python project and ask students to identify the role of each file and function

2. **Specifications**: Have students write specifications before coding (CRCN "Requirements expression" competency)

3. **Code review**: In pairs, each student reviews the other's code and suggests improvements

4. **Oral presentation**: The 5-minute defense prepares for the Baccalaureate oral exams (Grand Oral, NSI)

5. **Adaptation to a lower level**: Ask students to simplify the quiz for 6th graders (teaching transposition activity)

Trainer advice: The final project is the culmination of the module; it should be valued as a personal production. Support students in choosing their topic (pedagogical quiz, average calculator, educational game) and ensure they follow the development stages (specifications â†' design â†' implementation â†' testing). The 5-minute defense should be timed and evaluated according to clear criteria (code quality, pedagogical relevance, oral presentation).

Extensions (ENS Year 2)

- **OOP**: Refactor the quiz with classes (`Question`, `Quiz`, `Player`)
- **Web (Flask)**: Transform the quiz into a web application with forms and SQLite database
- **REST API**: Expose results via a JSON API
- **Cross-curricular project**: History-geography quiz, math exercises, language flashcards

Bibliography

- Python 3 Documentation: docs.python.org/3/tutorial/
- csv module: docs.python.org/3/library/csv.html
- Downey â€” *Think Python* (O'Reilly)
- NSI Program â€” eduscol.education.fr